

GOLANG

Living Textbook cho Software Engineering và DevOps/SRE

Edition nền móng

Được kiểm chứng với Go 1.27.1 - 20-09-2026

Markdown là nguồn gốc. PDF là bản đọc được, có thể tái tạo cục bộ.

Mục lục của edition này

Đây là edition nền móng. Mục lục tổng thể và thứ tự các phần tiếp theo được giữ trong `book/README.md` để có thể mở rộng mà không giả vờ rằng chúng đã được viết xong.

- Trước khi viết dòng Go đầu tiên
- Mở cửa vào Go
- Chương 1 — Đọc và viết một chương trình Go
- Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu
- Chương 3 — Mô hình dữ liệu và trách nhiệm thay đổi
- Chương 4 — Biên lỗi: để caller quyết định
- Chương 5 — Package là ranh giới
- Chương 6 — Thay đổi không sợ hãi
- Chương 7 — Dữ liệu đi vào và đi ra
- Chương 8 — Một race bắt đầu từ đâu

Trước khi viết dòng Go đầu tiên

Go không bắt đầu bằng câu hỏi “cú pháp nào còn thiếu?”. Nó bắt đầu bằng một vấn đề kỹ thuật rất cụ thể: đến năm 2007, máy tính đã nhanh hơn rất nhiều, nhưng việc xây và sửa một hệ thống phần mềm lớn vẫn bị kéo chậm bởi ngôn ngữ, build system và lượng bookkeeping mà lập trình viên phải mang theo.

Ở Google khi ấy, Robert Griesemer, Rob Pike và Ken Thompson làm việc giữa những codebase rất lớn, hệ thống nối mạng và phần cứng đang chuyển mạnh sang nhiều lõi. C++ và Java là lựa chọn phổ biến cho server; chúng đem lại những năng lực quan trọng, đồng thời tạo ra một lượng phức tạp đáng kể trong quá trình build, quản lý dependency và tổ chức chương trình. Ở phía khác, các ngôn ngữ động cho cảm giác phát triển nhanh hơn, nhưng đánh đổi một phần kiểm tra kiểu và đặc tính thực thi. Bài toán không phải là chọn một phe thắng phe còn lại. Bài toán là liệu một ngôn ngữ có thể đem hiệu quả biên dịch, hiệu năng và an toàn kiểu vào cùng một quy trình làm việc nhẹ hơn hay không.

Ngày 21 tháng 9 năm 2007, ba người bắt đầu phác thảo mục tiêu của một ngôn ngữ mới trên bảng trắng. Vài ngày sau, ngày 25 tháng 9, Pike đề xuất cái tên Go. Thiết kế ban đầu diễn ra song song với công việc khác; đầu năm 2008, Thompson đã có compiler thử nghiệm sinh mã C để thử các ý tưởng. Đến giữa năm, dự án đủ rõ để hướng tới compiler dùng trong thực tế. Ian Lance Taylor bắt đầu GCC front end vào tháng 5 năm 2008; Russ Cox gia nhập cuối năm ấy và giúp đưa prototype, library cùng tooling tiến gần một ngôn ngữ có thể dùng.



Hình 1 — Dòng thời gian không kể mọi release. Nó chỉ giữ các bước làm thay đổi lời hứa của Go với người viết phần mềm: công khai, ổn định, triển khai lại nền tảng, dependency rõ ràng và generics tương thích ngược.

Go được mở mã nguồn ngày 10 tháng 11 năm 2009. Ba năm đầu là giai đoạn ngôn ngữ còn thay đổi nhanh: nhóm thiết kế và cộng đồng thử, bỏ, sửa, rồi mới chốt một nền tảng ổn định. Go 1 phát hành ngày 28 tháng 3 năm 2012. Từ đây, Go 1 Compatibility Promise trở thành một phần của văn hóa dự án: code nguồn viết cho Go 1 được kỳ vọng tiếp tục build và chạy trong các bản Go 1 sau, với các ngoại lệ được nêu rõ. Lời hứa ấy làm thay đổi cách một feature được cân nhắc. Một bổ sung không chỉ cần hữu ích hôm nay; nó phải sống được cùng hệ sinh thái trong nhiều năm.

Go 1.5 là một cột mốc về implementation. Toolchain compiler chuyển từ C sang Go, runtime cũng trở thành Go kèm một phần assembly. Garbage collector được thiết kế lại để chạy phần lớn công việc đồng thời với chương trình; scheduler cho phép giá trị mặc định của GOMAXPROCS chuyển từ 1 sang số logical CPU. Đó là những thay đổi nằm dưới mặt ngôn ngữ, nhưng

chúng làm rõ mục tiêu ban đầu: một chương trình Go cần hợp với phần cứng nhiều lỗi, không bắt người viết phải tự gánh mọi chi tiết của runtime.

Những thay đổi lớn sau đó cũng đi theo nhịp tiến hóa thay vì đập đi làm lại. Go 1.11 đưa modules vào như một lựa chọn thay cho GOPATH, với versioning và package distribution tích hợp vào toolchain; mô hình này dần thành cách quản lý dependency quen thuộc. Go 1.18, phát hành năm 2022, đưa type parameters vào ngôn ngữ. Đến Go 1.27 năm 2026, generic methods nổi phần còn thiếu vào câu chuyện đó: method declaration có thể có type parameters riêng. “Go 2” vì thế không thành một nhánh rewrite tách rời; nhiều ý tưởng được thử, thảo luận và đưa dần vào dòng Go 1.x khi chúng minh được giá trị của chúng.

Một dòng họ ý tưởng, không phải một cây sao chép

Go thuộc nhiều dòng tư tưởng cùng lúc. Cú pháp cơ bản đặt nó gần họ C. Cách nhìn về declaration, package và sự rõ ràng cấu trúc nhận ảnh hưởng đáng kể từ Pascal, Modula và Oberon. Concurrency có một lịch sử dài hơn: CSP của Tony Hoare đi qua Newsqueak, Alef và Limbo trước khi channels trở thành một phần của Go. Những đường này giải thích vì sao một số ý tưởng nghe quen, chứ không làm Go thành bản sao của bất kỳ ngôn ngữ nào trong số đó.



Hình 2 — Nhánh CSP giải thích lịch sử của channels; nhánh C và Pascal/Modula/Oberon giải thích những điểm xuất phát khác. Thiết kế cuối cùng vẫn được tạo cho bài toán engineering mà Go nhắm đến.

Một ngôn ngữ được sinh ra cho codebase lớn không chỉ cần syntax gọn. Nó cần một cách làm việc chung quanh syntax: build nhanh, formatter, package/dependency rõ, compiler cho phản hồi sớm và tooling có thể phân

tích source. Đó là lý do Go thường được nói đến cùng `gofmt`, `go test` và `go command`, thay vì chỉ cùng một danh sách tính năng ngôn ngữ.

Thiết kế bằng cách từ chối

Trong giai đoạn đầu, nhóm sáng lập đặt nặng đồng thuận: một feature chỉ đi tiếp khi cả nhóm thấy nó thực sự cần. Câu hỏi không dừng ở “có thể thêm gì?”, mà còn là “có thể bỏ gì để hệ thống vẫn đủ sức làm việc?”. Kết quả là một bộ trade-off có chủ ý.

Một điều đáng nhớ: sự vắng mặt của inheritance hierarchy, operator overloading, exception-style error handling hay generics trong giai đoạn đầu không phải bản tuyên ngôn rằng các cơ chế ấy luôn tệ. Mỗi cơ chế có ích trong những ngữ cảnh khác. Go ưu tiên một mô hình dễ đọc, dễ phân tích, biên dịch nhanh và dễ giữ ổn định; cái giá là một số biểu đạt quen thuộc không có mặt hoặc đến muộn hơn.

Không có type hierarchy buộc người đọc đi tìm cây kế thừa; thay vào đó, interface và composition sẽ được nói kỹ khi ta đã đọc vững code cơ bản. Không có operator overloading giúp ý nghĩa của các toán tử ít thay đổi theo type. Error được trả như value thay vì theo đường exception-style. Và generics được hoãn lại cho đến khi thiết kế có thể đem giá trị đủ lớn so với độ phức tạp nó thêm vào type system và toolchain. Đây là các lựa chọn kỹ thuật có thể tranh luận, không phải luật đạo đức.

Nhìn lịch sử theo cách này giúp giải thích cảm giác đầu tiên khi đọc Go. Bạn sẽ thấy nhiều câu ngắn, ít keyword, declaration rõ và tooling hiện diện sớm. Những đặc điểm ấy là dấu vết của một mục tiêu: giảm ma sát cho người đang cùng người khác xây, đọc và sửa phần mềm lâu dài.

Sau gần hai thập kỷ tiến hóa, cách tốt nhất để gặp các quyết định ấy không phải đọc thêm lịch sử. Hãy đặt chúng trước mắt trong một tệp `.go` rất nhỏ.

Ghi chú kiểm chứng

1. Go Team. Frequently Asked Questions: Origins and Design. go.dev/doc/faq
2. Cox, Russ. Toward Go 2. 13 July 2017. go.dev/blog/toward-go2

3. Go Team. Go 1 and the Future of Go Programs. March 2012. go.dev/doc/go1compat
4. Gerrand, Andrew. Go 1.5 is released. 19 August 2015. go.dev/blog/go1.5
5. Bonventre, Andrew. Go 1.11 is released. 24 August 2018. go.dev/blog/go1.11
6. Go Team. Go 1.18 is released! 15 March 2022. go.dev/blog/go1.18
7. Go Team. Go 1.27 Release Notes. 2026. go.dev/doc/go1.27

Mở cửa vào Go

Bạn không cần thuộc ngay danh sách keyword để bắt đầu. Điều cần trước tiên là nhìn một tệp Go và biết: phần nào đặt tên cho chương trình, phần nào tạo giá trị, phần nào quyết định đường đi, và phần nào chỉ là một lời gọi hàm.

Cuốn sách này không đi bằng các mẫu bài học quá ngắn. Nó đi bằng những lần thay đổi cách nhìn: đọc được code trước, nhìn được giá trị chạy qua code sau, rồi mới đến những lúc nhiều goroutine, mạng, dữ liệu và production khiến trực giác ban đầu không còn đủ. Khi một chủ đề cần một lỗi, một trace hay một thí nghiệm để thấy rõ, chương đó sẽ bắt đầu từ bằng chứng ấy.

Cách đọc: đừng lướt qua code block. Mỗi lần gặp một đoạn ngắn, hãy thử nói thành lời nó tạo giá trị gì, giá trị đó được đặt tên ở đâu và điều gì sẽ xảy ra nếu đổi một token. Compiler là phản hồi nhanh nhất cho dự đoán của bạn.

Chương đầu không có mục tiêu “biết hết Go”. Mục tiêu nhỏ hơn, nhưng thiết yếu: bạn có thể đọc một chương trình console ngắn từ trên xuống, tự sửa nó, và hiểu vì sao một thay đổi là hợp lệ hoặc không hợp lệ.

Chương 1 — Đọc và viết một chương trình Go

Một tệp nguồn có hình học riêng: phần đầu đặt nó vào package, vài declaration đứng ở cấp tệp, rồi các block lồng vào nhau. Ta sẽ bắt đầu từ một chương trình rất nhỏ: nó nhận một mã trạng thái và in ra một nhãn mà người trực vận hành có thể đọc. Ví dụ đủ thực để có dữ liệu, lựa chọn và thông điệp, nhưng chưa che mất ngôn ngữ dưới một dự án.

```
package main

import "fmt"

const service = "checkout"

func classify(status int) string {
    if status >= 500 {
        return "failed"
    }
    return "healthy"
}

func main() {
    status := 503
    label := classify(status)
    fmt.Println(service, label)
}
```



Hình 3 — Hãy nhìn figure này như bản đồ định vị trước khi đọc chi tiết. Nó cho thấy package name, main package, function declaration và entry function là các khái niệm có quan hệ, nhưng không phải là một khái niệm duy nhất.

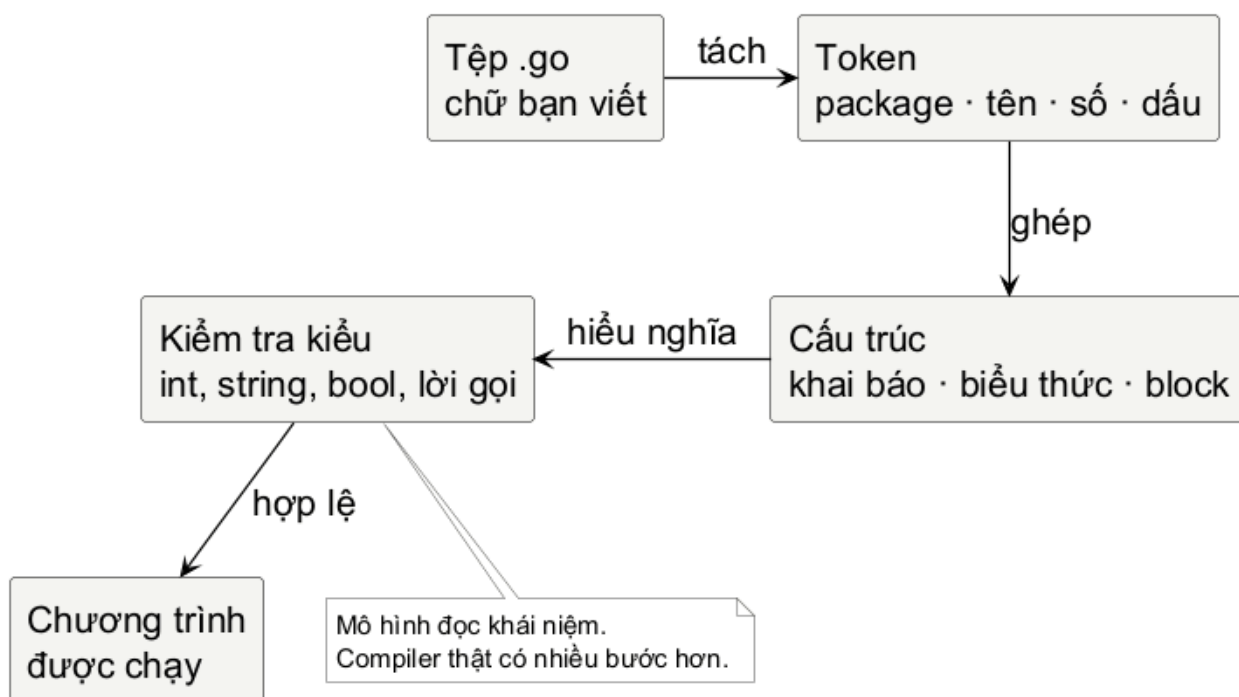
Chưa cần đọc từng dòng. Hãy chỉ theo ba câu hỏi: chương trình bắt đầu ở đâu, giá trị nào được tạo, và giá trị nào bị gửi ra màn hình? Câu trả lời là main, literal 503 cùng các giá trị trả về từ classify, và lời gọi Println.

Bản đồ của một tệp nguồn

Từ đầu xuống cuối, tệp trên có bốn vùng: package nói tệp thuộc chương trình nào; import cho phép dùng tên từ package khác; khai báo ở cấp tệp tạo const và hàm; cuối cùng, thân hàm main là nơi một chuỗi câu lệnh được chạy.

Dòng hoặc mảnh code	Nó là gì khi đọc	Nó có tác dụng gì
<code>package main</code>	package declaration	Đặt tệp vào package có tên main. Một executable hoàn chỉnh cần main package và func main không argument, không result.
<code>import "fmt"</code>	import declaration	Cho phép tham chiếu fmt từ standard library.
<code>const service = "checkout"</code>	declaration + literal	Gắn một tên không đổi với chuỗi.
<code>func classify(status int) string</code>	function declaration + signature	Đặt tên hàm, tham số kiểu int và kết quả kiểu string.
<code>status >= 500</code>	expression	Tính ra một giá trị bool.
<code>return "failed"</code>	statement	Kết thúc lời gọi hiện tại với một string.
<code>status := 503</code>	short variable declaration	Tạo biến status trong block main.
<code>fmt.Println(...)</code>	function call statement	Gọi hàm để có hiệu ứng in ra stdout.

Các nhãn này cho ta một cách chỉ vị trí khi compiler báo lỗi: lỗi có thể nằm trong một biểu thức, kiểu của một lời gọi, hay phạm vi của một tên thay vì chỉ là cảm giác mơ hồ rằng “code bị sai”.



Hình 4 — Đây là mô hình đọc, không phải sơ đồ nội bộ chính xác của Go compiler. Nó giải thích vì sao một dấu ngoặc, một tên không tồn tại và phép cộng sai kiểu thường bị bắt trước khi chương trình có cơ hội chạy.

Một tên chỉ có nghĩa trong phạm vi của nó

Ở dòng `const service`, `service` là identifier: một cái tên do người viết chọn. `fmt`, `Println`, `classify`, `status` và `label` cũng là identifier. Ngược lại, `package`, `import`, `const`, `func`, `if` và `return` là keyword; chúng dành riêng cho cấu trúc ngôn ngữ nên bạn không thể dùng chúng để tự đặt tên.

Tên không tự mang giá trị. Câu `status := 503` tạo một biến, rồi gán giá trị literal 503 vào biến đó. Trong cùng block, câu `label := classify(status)` tạo một biến khác, nhận kết quả mà hàm trả về. Tên tồn tại từ điểm khai báo đến cuối block bao quanh.

Phóng to một statement

Compiler không đọc `status := 503` như một câu tiếng Anh. Nó tách câu này thành các token có vai trò cú pháp, rồi ghép các token thành statement.

<code>status</code>	<code>:=</code>	<code>503</code>
identifier	operator	integer literal

Đây là một mức đọc khác với mức file hoặc function: identifier gọi lại một tên trong source, `:=` là operator khai báo-ngắn, còn `503` là integer literal. Khi nhìn code theo từng mức như vậy, một compiler message có chỗ để bám vào.

```
func main() {  
    status := 503  
    if status >= 500 {  
        label := "failed"  
        fmt.Println(label)  
    }  
    // fmt.Println(label) // không hợp lệ: label đã hết scope  
}
```

Khối `if` được tạo bởi cặp ngoặc nhọn. `label` chỉ hữu ích bên trong khối đó. Đây là một giới hạn có chủ ý: người đọc không phải lục lại toàn bộ tệp để đoán một tên có còn sống hay không.



Hình 5 — Scope không mô tả dữ liệu được lưu ở đâu trong bộ nhớ. Nó chỉ giới hạn nơi tên được phép được tham chiếu trong source.

Dừng lại để dự đoán: nếu bỏ comment ở dòng cuối, compiler phàn nàn về giá trị hay về tên? Câu trả lời là tên: `label` không còn được khai báo trong scope mà lời gọi `Println` đang đứng.

Làm chương trình lớn lên, mỗi lần một ý

Đừng viết đầy đủ từ đầu. Ta quan sát cùng một ý định khi nó nhận thêm một yêu cầu. Mỗi phiên bản giữ lại phần cũ có ích và chỉ thêm một khái niệm.

Literal: một kết quả cố định

```
package main

import "fmt"

func main() {
    fmt.Println("checkout healthy")
}
```

"checkout healthy" là string literal. Chương trình có một statement gọi hàm và không có quyết định nào. Nó đúng nếu thông điệp không bao giờ thay đổi.

Variable: dữ liệu thay đổi được đặt tên

```
func main() {
    service := "checkout"
    label := "healthy"
    fmt.Println(service, label)
}
```

Hãy nhìn một thay đổi trước khi học rule. Lần đầu, ta tạo status và cho nó giá trị. Lần sau, status đã có trong cùng block; ta chỉ thay giá trị của nó.

```
status := 503
status = 200
```

Vì dòng thứ hai không tạo một tên mới, nó dùng = thay vì :=. Short declaration chỉ dùng trong function body. Với nhiều tên ở vế trái, tên cũ có thể xuất hiện lại khi chúng đã có trong cùng block (hoặc parameter list tương ứng), type không đổi, và có ít nhất một non-blank variable mới. _ không được tính là tên mới. Đây là trường hợp tiện lợi khi bổ sung một kết quả mới mà vẫn giữ một biến đã có.

Muốn làm gì?	Viết gì?	Điều compiler cần biết
Tạo một biến mới và suy ra kiểu	<code>name := value</code>	Ít nhất một tên ở vế trái là mới trong scope hiện tại.
Gán lại biến đã có	<code>name = value</code>	Tên đã tồn tại và giá trị mới phù hợp kiểu.
Đặt giá trị không đổi ở cấp tệp	<code>const name = value</code>	Giá trị phải là hằng; không dùng <code>:=</code> .
Khai báo rõ kiểu trước	<code>var name Type = value</code>	Dùng khi kiểu hoặc zero value cần được nói rõ.

Expression: để chương trình tự kết luận

Literal 503 không tự nói rằng service thất bại. Ta cần một biểu thức tạo ra quyết định. `status >= 500` đọc là “status lớn hơn hoặc bằng 500”, và kết quả của nó là bool: true hoặc false.

```
func main() {
    status := 503
    if status >= 500 {
        fmt.Println("checkout failed")
    }
}
```

if nhận một biểu thức bool. Khi biểu thức là true, block của if được thực hiện; khi false, nó bị bỏ qua. Dấu ngoặc không chỉ để trang trí: chúng xác định chính xác nhóm câu lệnh phụ thuộc vào điều kiện.

Toán tử có thứ tự ưu tiên. Viết ngoặc khi chúng làm ý định rõ hơn thay vì bắt người đọc nhớ luật ưu tiên:

```
urgent := status >= 500 ||
    (status >= 400 && service == "checkout")
```

`&&` nghĩa là cả hai vế phải đúng; `||` nghĩa là ít nhất một vế đúng. Biểu thức bên trong ngoặc chạy trước phần `||`. Không cần biến mọi phép toán thành một câu đố — một tên như `urgent` thường dễ đọc hơn một điều kiện bị nén.

Function: đặt tên cho một quyết định có thể tái dùng

Khi điều kiện có ý nghĩa riêng, đưa nó vào hàm. Người đọc khi đó gặp một tên ở đúng mức trừu tượng thay vì phải giải mã lại cùng một điều kiện ở mọi nơi.

```
func classify(status int) string {  
    if status >= 500 {  
        return "failed"  
    }  
    return "healthy"  
}
```

Signature nói rõ hợp đồng tối thiểu: `classify` nhận một `int` và trả về một `string`. `status` là parameter, chỉ tồn tại trong lời gọi hàm. `return` không in gì cả; nó gửi một giá trị về nơi gọi. Vì thế câu `label := classify(status)` đọc theo thứ tự này: đọc `status` hiện có, gọi `classify` với giá trị đó, nhận `string` kết quả, rồi tạo `label`.

Cùng một quyết định, hai cách tổ chức

Nếu có nhiều trạng thái rời rạc, `switch` thường bộc lộ ý định tốt hơn một chuỗi `if`. Nó không “mạnh hơn”; nó chỉ đặt các nhánh cùng cấp cạnh nhau.

Đầu vào	Dạng phù hợp	Lý do đọc dễ hơn
Một ngưỡng: <code>status >= 500</code>	<code>if</code>	Điều kiện là một mệnh đề đúng/sai.
Vài giá trị cụ thể: 200, 404, 503	<code>switch</code>	Mỗi case là một lựa chọn cùng cấp.
Một thao tác lặp trên danh sách	<code>for</code> hoặc <code>range</code>	Ý định là làm lại, không phải chọn một nhánh.

```
func severity(status int) string {
    switch {
    case status >= 500:
        return "critical"
    case status >= 400:
        return "warning"
    default:
        return "normal"
    }
}
```

switch trên không có expression sau từ switch, nên từng case là bool expression. Go chạy case đầu tiên đúng rồi rời switch; không có fallthrough ngầm như một số ngôn ngữ khác. Thứ tự ở đây là dữ liệu của thiết kế: nếu đưa status >= 400 lên trước, 503 sẽ bị phân loại warning. Một compiler không thể bắt lỗi ý nghĩa đó cho bạn; đó là việc của người đọc và test.

Lặp là đi qua một tập giá trị

Một for có thể diễn tả vòng lặp điều kiện, nhưng range thường trực tiếp hơn khi ta muốn đi qua các phần tử của một collection.

```
statuses := []int{200, 503, 404}

for _, status := range statuses {
    fmt.Println(status, severity(status))
}
```

[]int là slice của int; cặp ngoặc nhọn tạo literal gồm ba phần tử. range cung cấp index và value. Tên _ bảo Go bỏ index đi vì ta không cần nó. Mỗi lượt, status là giá trị hiện tại; lời gọi severity dùng nó và Println in kết quả. Ta sẽ quay lại slice, bộ nhớ và aliasing ở chương sau — hiện tại chỉ cần thấy luồng đọc của vòng lặp.

Kiểu giới hạn những phép có nghĩa

Go suy ra nhiều kiểu từ literal, nhưng vẫn kiểm tra chúng chặt. `int` biểu diễn số nguyên; `bool` biểu diễn `true` hoặc `false`; `string` là một sequence of bytes không thể sửa trực tiếp. Text trong Go thường dùng UTF-8, và range trên `string` có semantics liên quan UTF-8/rune, nhưng `string` vẫn có thể chứa chuỗi byte không phải UTF-8 hợp lệ. Một biến có kiểu xác định; phép toán đòi các vế tương thích.

```
status := 503
message := "status: " + status // không hợp lệ
```

Toán tử `+` ở đây có thể nối hai `string` hoặc cộng hai số, nhưng không tự đoán cách trộn `string` và `int`. Chuyển đổi là một quyết định phải viết ra:

```
message := "status: " + fmt.Sprint(status)
```

`fmt.Sprint` tạo `string` để dùng cho hiển thị. Trong những ranh giới nghiêm túc hơn, ta sẽ phân biệt conversion với parsing và kiểm tra lỗi. Bây giờ điểm quan trọng là: kiểu giúp phát hiện một phép ghép vô nghĩa ngay tại nơi bạn viết nó.

Đọc compiler như một người cộng tác khó tính

Phần này là một cuộc điều tra nhỏ, không phải checklist lệnh. Trong lab `part1-reading-go`, giữ một bản đang chạy được. Sau đó tạo từng lỗi một, dự đoán trước, chạy compiler, và phân loại thông báo. Đừng sửa ngay khi vừa thấy chữ đỏ.

Case	Thay đổi có chủ ý	Dự đoán cần kiểm tra
A	Viết <code>status = 503</code> khi <code>status</code> chưa tồn tại	Lỗi thuộc về identifier hay giá trị?
B	Đổi <code>classify</code> nhận <code>string</code> nhưng caller vẫn đưa <code>status</code> kiểu <code>int</code>	Compiler nói về function hay type của argument?
C	Dùng <code>label</code> sau khi rời <code>if</code> block	Tên biến mất vì giá trị bị xóa hay vì scope kết thúc?

Kết quả dưới đây được chạy bằng Go 1.27.1 trên ba source file tối thiểu, mỗi case chỉ có một lỗi chủ ý. Dòng dài ở Case B được ngắt khi dàn trang; nội dung của compiler message được giữ nguyên.

```
[case-a]
# command-line-arguments
.\main.go:4:2: undefined: status

[case-b]
# command-line-arguments
.\main.go:9:15: cannot use status (variable of type int)
    as string value in argument to classify

[case-c]
# command-line-arguments
.\main.go:8:6: undefined: label
```

Case A là lỗi tên: compiler không thể gán cho một identifier chưa được khai báo. Case B đi xa hơn, vì tên tồn tại nhưng type của giá trị đưa vào không khớp parameter. Case C quay lại scope: `label` từng tồn tại trong block con, nhưng không thể được tham chiếu từ câu lệnh đứng sau block đó. Thông báo có thể khác đôi chút giữa phiên bản Go, còn ba loại lỗi thì vẫn là ba loại lỗi bạn cần nhìn ra.

Bài đọc cuối chương: không cần viết chương trình mới. Hãy nhìn hàm `severity`, dự đoán kết quả cho 200, 404 và 503, rồi viết một test table nhỏ cho ba dự đoán đó. Nếu test thất bại, lần theo branch được chọn thay vì sửa output một cách mù.

Đến đây, bạn đã có từ vựng để đọc phần lớn tệp Go nhỏ: `package` đưa tệp vào một đơn vị, `declaration` đặt tên, `expression` tạo giá trị, `statement` điều phối hiệu ứng, `block` giới hạn scope, và `function` biến một đoạn quyết định thành đơn vị có hợp đồng. Chương tiếp theo sẽ mở bằng một hiện tượng khó chịu hơn: hai biến trông độc lập, nhưng một thay đổi qua biến này lại hiện ra ở biến kia. Câu hỏi là chúng đã chia sẻ điều gì.

Chương 2 — Khi một bản sao vẫn chia sẻ dữ liệu

Chạy đoạn này trước khi cố giải thích nó. Nếu bạn đoán a và b khác nhau sau dòng `b[0] = 99`, đó là một dự đoán rất hợp lý - và cũng là cánh cửa vào phần này.

```
a := []int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a)
fmt.Println(b)
```

Kết quả từ lab part2-values là:

```
[99 20 30]
[99 20 30]
```

Đừng ghi nhớ riêng một câu “slice nguy hiểm”. Thay vào đó, ta sẽ lần ngược từ kết quả: Go đã copy cái gì khi chạy `b := a`, và dữ liệu nào không được copy?

Một baseline để dự đoán

Với một `int`, assignment tạo ra một giá trị độc lập. Cả a và b đều có giá trị riêng của chính chúng.

```
a := 10
b := a
b = 99

fmt.Println(a, b) // 10 99
```

Go đã copy value của a vào b. Sau đó b nhận value mới; a không có lý do gì để đổi. Đây là quy tắc nền: assignment copy value. Điều còn thiếu là một value có thể chứa gì.

Array giúp ta kiểm tra quy tắc ấy với nhiều phần tử. Độ dài là một phần của type, nên `[3]int` và `[4]int` là hai array type khác nhau. Khi array được gán, toàn bộ array value được copy.

```
a := [3]int{10, 20, 30}
b := a
b[0] = 99

fmt.Println(a) // [10 20 30]
fmt.Println(b) // [99 20 30]
```



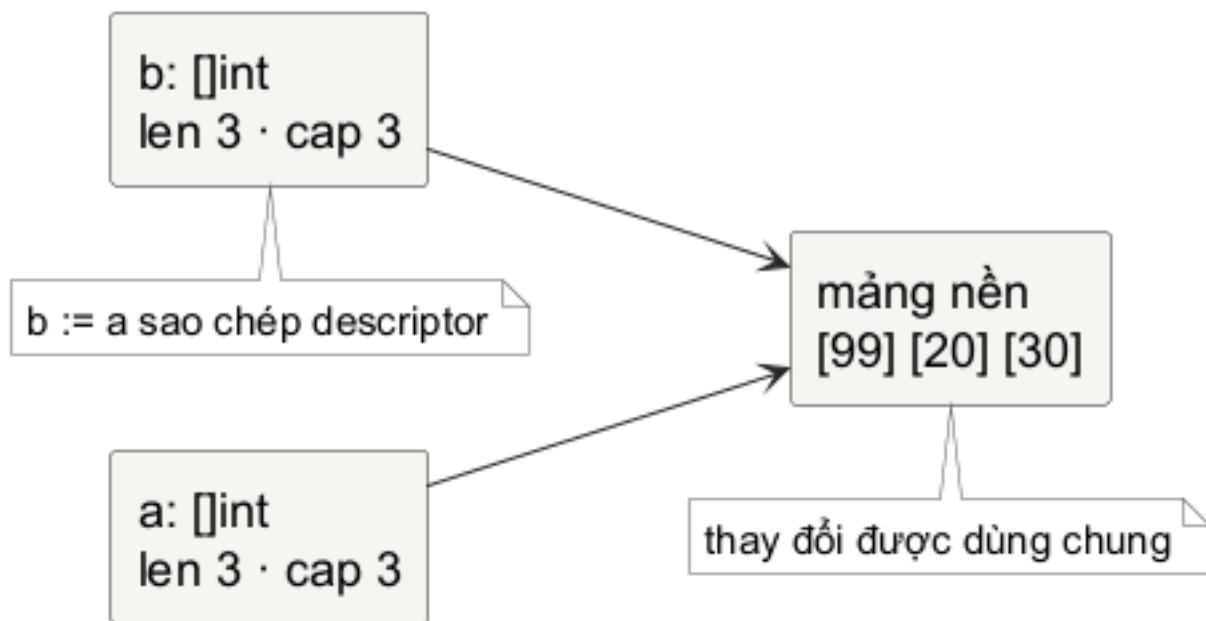
Hình 6 — Assignment của array copy các phần tử. Việc thay `b[0]` chỉ chạm vào array `b`.

Đến đây, trực giác “copy nghĩa là độc lập” vẫn đúng. Sự thay đổi xuất hiện chỉ khi ta bỏ một ký tự trong type: `[3]int` trở thành `[]int`.

Khoảnh khắc slice đổi kết quả

```
a := []int{10, 20, 30}
b := a
b[0] = 99
```

`[]int` không phải array không ghi độ dài. Nó là slice type. Một slice value mô tả một đoạn liên tiếp của underlying array. Ngôn ngữ định nghĩa slice value có length, capacity và một reference tới underlying array. Đó là mental model hữu ích cho semantics; nó không phải lời hứa rằng mọi Go implementation phải dùng đúng một struct hay ABI như hình.



Hình 7 — `b := a` copy slice value, gồm của số nhìn vào dữ liệu, len và cap. Cả hai descriptor vẫn dẫn tới cùng underlying array, nên `b[0]` thay đổi phần tử mà `a` cũng nhìn thấy.

Vì vậy hai câu sau cùng đúng, và không mâu thuẫn:

Câu nói	Đúng ở đâu?
Assignment copy value.	<code>b</code> nhận một slice value riêng: đổi len của <code>b</code> bằng <code>reslice</code> không tự đổi len của <code>a</code> .
Hai slice có thể cùng đổi dữ liệu.	Slice value có thể cùng reference một underlying array; mutation một element hiện ra qua mọi slice cùng nhìn element đó.

Bảng 1 — Copy value và chia sẻ storage là hai tầng khác nhau của cùng một phép gán.

Từ aliasing dùng cho tình huống nhiều đường tham chiếu đến cùng dữ liệu. Nó không phải lỗi tự thân. Aliasing giúp cắt một cửa sổ trên buffer mà không copy cả buffer. Nó thành bug khi một người đọc tưởng mình có dữ liệu riêng nhưng thực tế lại đang sửa vùng dùng chung.

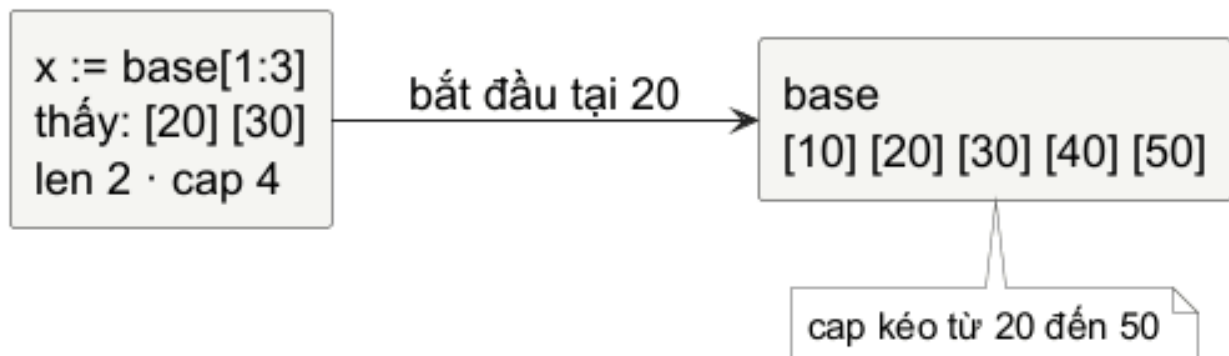
Cửa sổ, length và capacity

Đặt một slice lên một slice khác để thấy descriptor thực sự đang mô tả gì.

```
base := []int{10, 20, 30, 40, 50}
x := base[1:3]
```

```
fmt.Println(x)          // [20 30]
fmt.Println(len(x))     // 2
fmt.Println(cap(x))     // 4
```

x bắt đầu ở phần tử 20 của base và kết thúc trước 40. Vì thế length là 2. Capacity đếm số phần tử từ điểm bắt đầu của x đến cuối backing array mà x hiện có thể mở rộng tới; ở đây là 20, 30, 40, 50, nên là 4.



Hình 8 — x nhìn thấy hai phần tử đầu của cửa sổ. cap(x) không nói x đang có bốn phần tử; nó nói x có thể được reslice hoặc append trong phạm vi backing array hiện tại.

Hãy tự dự đoán trước khi chạy:

```
base := []int{1, 2, 3, 4}
a := base[:2]
b := base[1:3]
a[1] = 99
```

```
fmt.Println(base)
fmt.Println(a)
fmt.Println(b)
```

Kết quả là [1 99 3 4], [1 99], [99 3]. a[1] và b[0] đều chỉ vào phần tử base[1]. Hình vẽ không thay thế được việc lần index: nó chỉ cho biết vì sao có thể có một phần tử chung để lẫn.

append là một phép thử về capacity

append không hứa luôn dùng lại array cũ, cũng không hứa luôn tạo array mới. Nó trả về resulting slice. Khi backing array còn chỗ, implementation có thể dùng lại; khi capacity không đủ, append phải cấp underlying array mới đủ lớn cho kết quả. Vì return value mang descriptor có thể đã đổi, viết `s = append(s, value)` là dạng thông thường.

```
s := make([]int, 2, 4)
s[0], s[1] = 10, 20
old := s

s = append(s, 30)
s[0] = 99

fmt.Println(old) // [99 20]
fmt.Println(s)   // [99 20 30]
```

Ở đây `s` còn capacity. `old` và `s` có length khác nhau, nhưng vẫn có thể cùng nhìn hai element đầu của backing array. Ngược lại, full slice expression bên dưới giới hạn capacity của view ở 2, nên append buộc phải có storage khác cho grown:

```
old := []int{10, 20}
s := old[:2:2]
grown := append(s, 30)
grown[0] = 99

fmt.Println(old)    // [10 20]
fmt.Println(grown)  // [99 20 30]
```



Hình 9 — Hình mô tả hai semantics mà code phải chịu được. Đừng suy luận allocation từ một capacity tĩnh cở; nếu code cần một kết quả riêng, hãy tạo bản copy có chủ ý.

Một aliasing bug có người dùng thật

Một function “chỉ chuẩn bị preview” đôi khi phá dữ liệu của caller.

```
func redactPreview(fields []string) {
    fields[0] = "***"
}

requestFields := []string{"api-key", "region"}
redactPreview(requestFields)
fmt.Println(requestFields) // [*** region]
```

Parameter fields nhận một slice value mới, nhưng descriptor ấy vẫn reference storage của requestFields. Việc đổi element là mutation shared storage. Nếu mục tiêu thật sự là tạo preview độc lập, copy phần tử sang một slice mới trước:

```
func redactedCopy(fields []string) []string {
    preview := make([]string, len(fields))
    copy(preview, fields)
    preview[0] = "***"
    return preview
}
```

copy sao chép tối đa số phần tử vừa với destination và trả về số phần tử đã copy. Ở đây make tạo backing array mới, nên preview độc lập ở từng slice elements. Đổi lấy independence là allocation và chi phí copy; với buffer lớn hoặc đường xử lý nóng, đó là một quyết định cần có lý do. Với một preview cần an toàn khỏi mutation, nó diễn đạt đúng intent.

Thực hành - bug hunt có kiểm chứng. Mở labs/part2-values trong VS Code và chạy riêng TestRedactedCopyDoesNotMutateInput trước khi sửa source. Sau đó thay phần tạo preview bằng `preview := fields`. Đừng đoán bằng mắt: chạy lại test để thấy nó báo input đã bị mutate. Khôi phục code, xóa thân redactedCopy, rồi tự viết lại từ contract của test mà không nhìn implementation cũ. Đây là một bước nhỏ, nhưng nó buộc mental model “copy slice value vẫn có thể chia sẻ storage” đi qua một failure thật.

Đáp án — chỉ đọc sau khi đã tự làm. `preview := fields` chỉ copy slice descriptor, nên `preview[0]` vẫn sửa phần tử chung với `requestFields`. Bản đúng cần một backing array mới, chẳng hạn make rồi copy. Test phải giữ cả hai assertion: preview bị redacted và input không đổi; bỏ assertion thứ hai là bỏ mất contract ownership.

String: bytes trước, text sau

String cũng là value, nhưng nó có luật riêng: string là immutable sequence of bytes. len đếm byte; index trả về byte. Text thường là UTF-8, song Go không yêu cầu mọi string phải chứa UTF-8 hợp lệ.

```
text := "Việt"
fmt.Println(len(text)) // 6

for index, value := range text {
    fmt.Println(index, value)
}
```

Lab chạy đoạn này với kết quả:

```
6
0 86
1 105
2 7879
5 116
```

V và i mỗi chiếm một byte. ệ được mã hóa bằng ba byte UTF-8, vì vậy rune tiếp theo bắt đầu ở byte index 5. range trên string decode UTF-8: giá trị thứ hai là rune, thường dùng để biểu diễn Unicode code point. byte là alias của uint8; rune là alias của int32. Code point vẫn không đồng nghĩa với “một ký tự người dùng nhìn thấy”: một grapheme có thể gồm nhiều code point. Ta chưa cần giải quyết segmentation ở đây; chỉ cần không nhầm len với số ký tự hiển thị.

String không cho gán trực tiếp text[0] = 'v'. Khi cần dữ liệu byte có thể sửa, chuyển sang []byte, thao tác, rồi tạo string mới nếu cần. Việc chuyển đổi đó diễn đạt một thay đổi ý định: từ text immutable sang buffer mutable.

Pointer là câu hỏi khác

Sau slice, pointer dễ bị giải thích sai như “slice nhưng rõ hơn”. Pointer là một value trực tiếp reference một variable khác; slice là một descriptor cho segment của underlying array. Chúng có thể cùng dẫn tới sharing, nhưng chúng trả lời hai câu hỏi khác nhau.

```
count := 10
p := &count
*p = 99
fmt.Println(count) // 99
```

Ở đây `&count` tạo pointer tới variable `count`, còn `*p` truy cập variable đó. Pointer sẽ có chương riêng khi ta cần thiết kế API quanh mutation trực tiếp và nil. Trước khi đến đó, hãy giữ model vừa xây: luôn hỏi value nào được copy, và value ấy có reference tới storage hay variable nào không.

Nếu câu trả lời là “không”, assignment thường tạo independence như `int` và `array`. Nếu câu trả lời là “có”, aliasing có thể là công cụ tốt hoặc một bug - tùy việc bạn có làm rõ ownership và mutation hay không.

Một quyết định ở ranh giới API

Khi một hàm nhận hoặc trả về slice, câu hỏi thiết kế không phải là “Go có copy không?” mà là: **ai được phép sửa storage này, và trong khoảng thời gian nào?** Một hàm lọc dữ liệu có thể trả về một view để tránh allocation; khi đó caller cần biết view ấy còn dùng chung buffer. Một hàm dựng response hoặc preview thường nên trả về dữ liệu độc lập, vì caller không có lý do để đoán một thay đổi sau đó sẽ đi ngược vào input.

Không có đáp án mặc định miễn phí. Copy tốn allocation và thời gian; chia sẻ tiết kiệm chúng nhưng làm contract về ownership quan trọng hơn. Trước khi tối ưu, hãy viết test cho hành vi mutation mà API hứa. Sau đó, khi đọc một bug khó hiểu, lần theo ba bước: value nào được gán, value đó reference storage nào, và ai còn có thể chạm storage ấy. Đó là cách biến “slice lạ quá” thành một cuộc điều tra có thể kiểm chứng.

Chương 3 — Mô hình dữ liệu và trách nhiệm thay đổi

opsprobe sẽ dần trở thành một CLI kiểm tra trạng thái dependency. Trước khi nó gọi HTTP hay đọc file config, nó đã có một bài toán nhỏ hơn: làm sao để biểu diễn trạng thái của một service mà người đọc không phải tự ghép bốn biến rời rạc trong đầu?

```
serviceName := "billing"
port := 8080
healthy := true
retries := 0
```

Bốn biến này đi cùng nhau. Chúng mô tả một service, nhưng compiler chưa biết điều đó. Càng nhiều hàm nhận chúng rời rạc, càng dễ đổi nhầm thứ tự, quên truyền một field, hoặc cập nhật một bản sao mà tưởng đã cập nhật trạng thái chung. Ta sẽ refactor dần mô hình này. Mỗi lần thay đổi, ta dừng ở cùng một câu hỏi: **function nhận value nào, và value ấy mở đường tới dữ liệu nào?**

Bug đầu tiên: bonus đi đâu mất?

Hãy tạm quên service. Một ví dụ nhỏ hơn cho thấy vấn đề rõ nhất:

```
balance := 100

func addBonus(x int) {
    x += 20
}

addBonus(balance)
fmt.Println(balance) // 100
```

balance không đổi vì lời gọi hàm cũng là một phép truyền value. Khi gọi `addBonus(balance)`, function nhận một parameter `x` có value 100. Câu `x += 20` thay value trong variable `x`; nó không quay lại gán cho variable

`balance` của caller. Đây không phải một ngoại lệ của `int`, mà là nền cho mọi signature trong chương này.



Hình 10 — Lỗi gọi `addBonus(balance)` copy value 100 vào parameter `x`. Sơ đồ biểu diễn semantics của variable và value, không khẳng định vị trí vật lý của chúng trong RAM.

Nếu function cần tạo một value mới, cách trên rất tốt: giá trị đi vào, kết quả đi ra bằng `return`. Ví dụ, `withBonus(balance)` có thể trả 120, và caller chọn có gán kết quả đó cho `balance` hay không. Nhưng một số thao tác có ý nghĩa là thay đổi chính variable caller đã đưa vào. Lúc ấy cần biểu đạt đường đi tới variable đó một cách rõ ràng.

Pointer: con đường tới một variable

Với operand addressable `x` có type `T`, `&x` tạo pointer value type `*T` trỏ tới variable `x`. Nếu `p` có type `*T`, `*p` là variable type `T` mà pointer đó trỏ tới. Hai câu này phân biệt năm thứ thường bị gộp làm một:

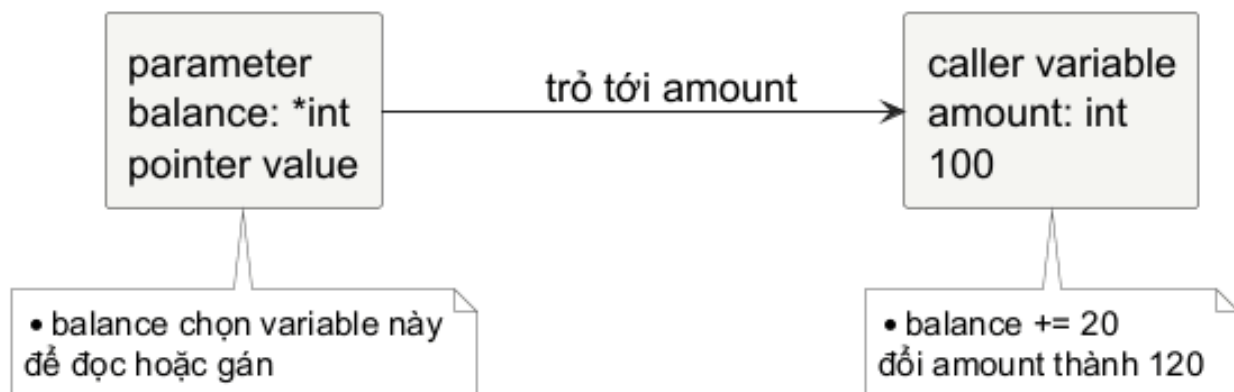
- **variable** là nơi chương trình có thể đọc hoặc gán value, như `balance`.
- **value** là dữ liệu đang được lưu, như 100.
- **address** là kết quả của phép lấy địa chỉ `&balance` trong ngữ cảnh Go.
- **pointer value** là value có type như `*int`; nó có thể trỏ tới một variable hoặc là `nil`.
- **pointee** là variable mà pointer hiện trỏ tới.

Ta không cần giả định variable ấy nằm cố định trên stack hay heap. Đó là quyết định implementation có thể thay đổi. Điều cần dùng khi đọc code là quan hệ ngôn ngữ: pointer value có đang trỏ tới một variable hợp lệ hay không.

```
func addBonus(balance *int) {
    *balance += 20
}

amount := 100
addBonus(&amount)
fmt.Println(amount) // 120
```

Ở caller, `&amount` tạo pointer tới variable `amount`. Parameter `balance` vẫn là một value được copy vào function - lần này value đó là pointer. Khi `*balance += 20` chạy, dereference `*balance` chọn chính variable `amount` làm đích gán. Bản sao của pointer không tạo bản sao của pointee.



Hình 11 — `balance` trong function là pointer value được truyền theo value. `*balance` truy cập pointee; sơ đồ cố ý không dùng địa chỉ số hay stack/heap để tránh biến mental model thành chi tiết runtime.

Phép `&` không dùng được với mọi expression. Go yêu cầu operand addressable: variable, dereference pointer, index của slice, field của struct addressable, index của array addressable, hoặc composite literal là những trường hợp quan trọng ở giai đoạn này. Map index không addressable - ta sẽ thấy lý do thực hành của giới hạn đó khi đưa map vào mô hình service.

Nil là thiếu pointee, không phải "pointer rỗng vô hại"

Zero value của pointer type là `nil`. `nil` không trở tới variable nào, nên dereference nó gây run-time panic.

```
var bonus *int

if bonus == nil {
    fmt.Println("chưa có bonus")
    return
}
*bonus += 20
```

Dereference `*bonus` cần một pointee để chọn làm đích gán; `nil` không có pointee, nên dereference nó panic. Kiểm tra `nil` là một phần của contract: function này có chấp nhận "không có giá trị" hay caller phải luôn truyền pointer hợp lệ? Đừng thêm `nil` check theo thói quen. Hãy quyết định và thể hiện điều đó qua API, test, hoặc lỗi trả về khi có lý do.

Gom state thành một value có tên

Quay lại bốn biến rời rạc. Một struct gom chúng thành một value mà domain có thể gọi tên:

```
type Service struct {
    Name      string
    Port      int
    Healthy   bool
    Retries   int
}

billing := Service{
    Name:      "billing",
    Port:      8080,
    Healthy:   true,
}
```

`type Service` khai báo một named type; phần thân struct khai báo các field và type của chúng. Struct literal đặt value cho các field được nêu. `Retries` không xuất hiện trong literal nên nhận zero value của `int`, là `0`. Zero value không thay thế validation: một `Service{}` hợp lệ về type nhưng chưa chắc hợp lệ cho một service thật. Trong domain này, port `0` có thể nghĩa là "chưa cấu hình".

Struct không phải class thiếu method. Trước hết nó là một value có nhiều phần mang cùng một ý nghĩa. Field access chỉ đọc hoặc gán một phần của value:

```
billing.Retries++  
fmt.Println(billing.Name, billing.Retries) // billing 1
```

Một struct cũng được copy khi gán hoặc truyền làm argument. Với các field trong Service hiện tại, copy tạo các field độc lập:

```
candidate := billing  
candidate.Healthy = false  
  
fmt.Println(billing.Healthy) // true  
fmt.Println(candidate.Healthy) // false
```



Hình 12 — Assignment copy struct value hiện tại theo từng field. Nếu một field về sau là slice, map hoặc pointer, struct vẫn được copy nhưng field đó có thể mang theo một reference; lúc ấy phải phân tích tiếp value của field thay vì gán nhãn "struct luôn độc lập".

Nested struct hữu ích khi một phần của domain tự nó đã có ý nghĩa. Ví dụ, host và port thường đi cùng nhau hơn là hai field phẳng:

```

type Endpoint struct {
    Host string
    Port int
}

type Service struct {
    Name      string
    Endpoint Endpoint
    Healthy   bool
    Retries   int
}

```

Việc nesting không tự động làm thiết kế tốt hơn. Nó có ích khi tạo ra một khái niệm đọc được, như `Endpoint`; gom chỉ để tránh thấy nhiều field thường làm model khó tìm hơn.

Khi function sửa một bản copy

Ta muốn `opsprobe` đánh dấu service không khỏe sau một lần probe thất bại. Phiên bản đầu tiên trông rất có lý:

```

func markUnhealthy(service Service) {
    service.Healthy = false
    service.Retries++
}

markUnhealthy(billing)
fmt.Println(billing.Healthy, billing.Retries) // true 0

```

Không có bug bí ẩn ở đây. `billing` được copy vào parameter `service`; function thay đổi struct copy rồi return. Caller còn giữ value cũ. Nếu đây là một transform thuần, ta có thể trả `Service` mới. Nếu ý định là mutate state caller đang quản lý, signature cần nói điều đó:

```
func markUnhealthy(service *Service) {
    service.Healthy = false
    service.Retries++
}

markUnhealthy(&billing)
fmt.Println(billing.Healthy, billing.Retries) // false 1
```

T và *T không có nghĩa "immutable" và "mutable". Một local value `Service` vẫn có thể bị gán field; một pointer parameter có thể chỉ được đọc. Khác biệt là một T parameter mang struct value riêng, còn một *T parameter mang khả năng truy cập pointee của caller. Chọn pointer khi function phải thay đổi pointee, khi biểu diễn sự vắng mặt bằng `nil` là một phần hữu ích của domain, hoặc khi API có lý do rõ ràng khác. Chọn value khi function cần snapshot cục bộ, transform rồi return, hoặc sharing sẽ làm contract khó hiểu hơn.

Cùng một lời gọi, bảy value khác nhau

Đến đây có thể đọc nhiều signature bằng một quy tắc duy nhất: Go truyền argument theo value. "Theo value" không trả lời xong câu hỏi; ta còn phải nhìn value đó chứa dữ liệu nào hoặc reference nào.

Parameter	Value được copy khi gọi	Thay đổi từ function có hiện ra ở caller?
<code>x int</code>	số nguyên	Không, trừ khi return và caller gán kết quả.
<code>a [3]int</code>	toàn bộ array value	Không, các element của bản copy bị sửa riêng.
<code>s []int</code>	slice value, gồm reference tới array nền	Sửa element có thể hiện ra; reslice parameter không tự đổi lên caller.
<code>m map[string]int</code>	map value, có reference tới map data	Ghi/xóa entry hiện ra qua map value cùng map data.
<code>p *int</code>	pointer value	<code>*p = ...</code> sửa pointee của caller nếu pointer hợp lệ.
<code>v Service</code>	các field của struct value	Không với các field value; xem từng field nếu chúng chứa reference.
<code>p *Service</code>	pointer value	Sửa field qua p sửa pointee của caller nếu pointer hợp lệ.

Bảng 2 — Mỗi lời gọi đều copy argument value vào parameter; khả năng chia sẻ nằm trong value được copy.

Bảng này không phải bảng để học thuộc type. Nó là cách lần một lời gọi: parameter mới nhận bản copy nào; bản copy ấy có dẫn đến dữ liệu chung hay variable chung không; expression nào là đích mutation? Đó cũng là lý do value receiver và pointer receiver sẽ được học sau cùng methods, thay vì biến dấu `*` thành một nghi thức.

Bài tập - sửa đúng contract: Viết `recordFailure(service Service)` sao cho caller nhận được `Healthy == false` và `Retries` tăng một. Trước khi nhìn đáp án, chọn một trong hai contract: function return một `Service` mới, hoặc function nhận `*Service` và mutate pointee.

Đáp án. Nếu caller quản lý cùng một state và function được đặt tên như một command, pointer làm ý định trực tiếp:

```
func recordFailure(service *Service) {
    service.Healthy = false
    service.Retries++
}

recordFailure(&billing)
```

Một biến thể value vẫn đúng khi API diễn đạt transform:

```
func failedCopy(service Service) Service {
    service.Healthy = false
    service.Retries++
    return service
}

billing = failedCopy(billing)
```

Hai phiên bản khác nhau ở trách nhiệm của caller. Phiên bản pointer cho phép function tác động lên state đã có; phiên bản return khiến việc thay state nằm rõ ở câu gán của caller. Không có "bản nhanh hơn" mặc định quan trọng hơn contract ở đây.

Thực hành - một bug vẫn compile. Trong `labs/part3-data-models`, mở `main_test.go` trước. Ở `recordProbe`, tạm bỏ câu `registry[name] = service`, rồi chạy riêng

`TestRecordProbeUpdatesExistingMapEntry`. Code vẫn compile vì local struct copy được sửa hợp lệ; test mới chỉ ra entry trong map không hề nhận value mới. Khôi phục bằng đường mutation mà anh đã lần, không bằng cách sửa expectation. Sau đó xóa thân `recordFailure` và tự dựng lại nó từ `TestRecordFailureMutatesPointee`.

Đáp án — chỉ đọc sau khi đã tự làm. Map lookup trả một `Service` value. Sửa local `service` chưa chạm map data; phép gán ngược vào `registry[name]` mới là mutation của registry. `recordFailure` cần nhận `*Service` và sửa pointee, vì test đòi caller quan sát cùng state thay đổi.

Nhiều service cần lookup, không cần thêm biến

Một Service model một service. Khi opsprobe giữ nhiều service theo tên, ta không muốn billing, search, checkout thành các variable độc lập.

Map diễn đạt một quan hệ key/value:

```
services := map[string]Service{
    "billing": {
        Name:    "billing",
        Port:    8080,
        Healthy: true,
    },
}

billing, found := services["billing"]
fmt.Println(found, billing.Port) // true 8080
```

string là key type; Service là element type. found là kết quả thứ hai của map lookup, thường gọi là comma-ok. Nó phân biệt key không có mặt với element có zero value. Điều này quan trọng với counter:

services["missing"].Retries không phải cách hợp lệ để đọc field, nhưng với map[string]int, lookup key thiếu và key có value 0 đều cho 0 nếu chỉ nhận một result.

```
attempts := map[string]int{"billing": 0}

count, found := attempts["billing"]
fmt.Println(count, found) // 0 true

count, found = attempts["search"]
fmt.Println(count, found) // 0 false
```

Map literal tạo map đã sẵn sàng ghi. make(map[string]Service) cũng tạo map sẵn sàng ghi. Ngược lại, zero value của map là nil: đọc từ nil map cho zero value, range qua nó có zero iteration, nhưng gán entry vào nil map sẽ panic. Một map nhận từ caller vì thế cần contract rõ như pointer: caller có luôn khởi tạo nó không, hay function chịu trách nhiệm tạo nó trước khi

ghi?

```
var services map[string]Service
fmt.Println(services["billing"].Name) // ""

// services["billing"] = billing // panic: assignment to entry in nil map
services = make(map[string]Service)
services["billing"] = billing
```

`delete(services, "billing")` xóa một entry; xóa key không có mặt là an toàn. Nhưng range map không có thứ tự được chỉ định. Nếu CLI cần output ổn định, hãy lấy key, sort chúng, rồi in theo thứ tự đã chọn - đừng coi thứ tự đang tình cờ xuất hiện là contract của map.

Map copy value, chia sẻ map data

Map không phải pointer và ta không cần mô tả hash table internals để dùng nó đúng. Assignment và call copy map value. Nếu hai map value cùng reference map data, ghi qua một value hiện ra qua value kia:

```
registry := map[string]int{"billing": 1}
alias := registry
alias["billing"]++

fmt.Println(registry["billing"]) // 2
```



Hình 13 — Assignment copy map value. Khi value ấy reference cùng map data, mutation entry qua `alias` hiện ra khi đọc qua `registry`; hình là conceptual diagram, không mô tả hash table implementation.

Một chi tiết làm map khác slice và array khi update struct value: map index không addressable. Vì vậy đoạn này không hợp lệ:

```
// services["billing"].Healthy = false // không hợp lệ
```

`services["billing"]` cho một `Service` value để đọc; nó không cho một `variable` mà ta có thể lấy địa chỉ field hay gán field trực tiếp. Cách `value-oriented` là lấy value ra, đổi, rồi gán lại:

```
service := services["billing"]
service.Healthy = false
services["billing"] = service
```

Nếu domain muốn registry quản lý các `*Service`, thì mutation qua pointee có thể trực tiếp hơn, nhưng đổi lại registry có thể chứa nil pointer và ownership cần được làm rõ:

```
services := map[string]*Service{
    "billing": &Service{Name: "billing", Port: 8080, Healthy: true},
}

services["billing"].Healthy = false
```

Đây không phải lời mời chọn `map[string]*Service` mặc định. Với value nhỏ như `Service` hiện tại, `map[string]Service` và "lấy - sửa - gán lại" thường bộc lộ điểm mutation rõ hơn. Pointer trong map chỉ đáng dùng khi nó giải quyết một contract thực: object identity dùng chung, mutation phải đi qua nhiều nơi, hoặc value lớn và đã có lý do đo đạc/thiết kế.

Bài tập - đọc zero value: Một probe trả về `latencyByService` `map[string]int`. Viết code phân biệt `billing` có latency 0 với `billing` chưa có kết quả. Sau đó xóa kết quả của `billing` mà không cần kiểm tra key có tồn tại.

Đáp án. Comma-ok mang thông tin hiện diện; `delete` tự xử lý key thiếu:

```
latency, found := latencyByService["billing"]
if !found {
    fmt.Println("chưa có kết quả")
} else {
    fmt.Println(latency)
}

delete(latencyByService, "billing")
```

Một update có trách nhiệm trong registry

`opsprobe` chưa cần network để hưởng lợi từ model này. Một update value-oriented cho registry không cần pointer: lấy struct value ra khỏi map, sửa bản local, rồi gán nó trở lại entry. Map parameter được truyền theo value, nhưng map value ấy vẫn mở đường tới map data của caller; chính phép gán cuối là mutation có chủ ý lên registry chung.

```

func recordProbe(registry map[string]Service, name string, healthy bool)
    service, found := registry[name]
    if !found {
        return false
    }

    service.Healthy = healthy
    if !healthy {
        service.Retries++
    }
    registry[name] = service
    return true
}

```

Signature này nói rõ ba vai: `registry` là map value để lookup và gán entry; `name` là snapshot string; `healthy` là snapshot bool. `service` là struct copy lấy từ entry. Nếu key không có, function không bịa ra service mới mà trả `false`; nếu có, nó ghi value đã sửa về map. Khi công cụ bắt đầu đọc config và probe endpoint ở phần sau, cách đọc signature này sẽ giúp ta biết dữ liệu nào được chia sẻ, dữ liệu nào được snapshot, và mutation nào có thể lặp lại.

Sau chương này, một signature không còn chỉ là tên type. Nó cho biết ta đang đưa vào function một snapshot, một cửa sổ tới storage dùng chung, một map value dẫn tới map data, hay một pointer mở đường tới variable caller. Khi cần debug mutation, hãy viết ra trước/sau lời gọi: variable nào tồn tại ở caller, parameter nào xuất hiện ở function, và expression nào thực sự là đích gán.

Khi hành vi thuộc về service

`recordProbe(registry, name, healthy)` đã làm đúng việc, nhưng khi `opsprobe` có thêm nhiều thao tác quanh một service, chữ ký bắt đầu lặp lại cùng một chủ thể. Một hàm cần tạo nhãn cho service, một hàm khác ghi kết quả probe, hàm nữa kiểm tra cấu hình. Lúc này method giúp đặt hành vi cạnh type mà nó đọc hoặc thay đổi; nó không thay đổi quy tắc value semantics đã xây.

```
func (service Service) Summary() string {
    return service.Name
}

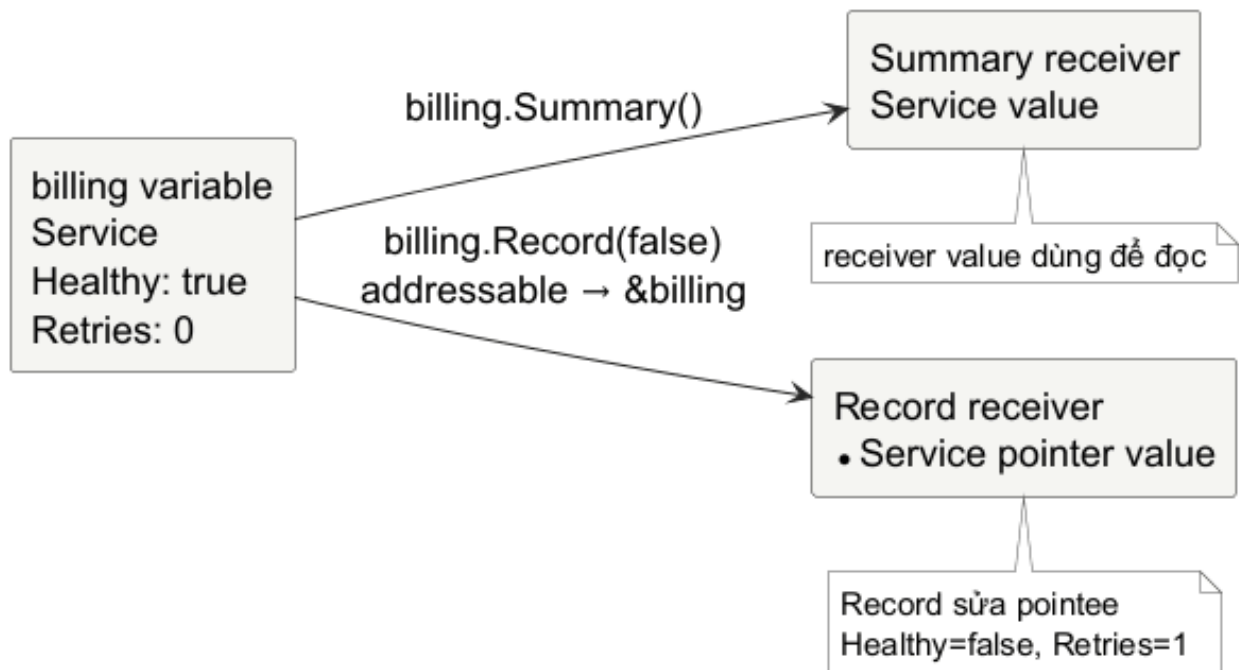
func (service *Service) Record(healthy bool) {
    service.Healthy = healthy
    if !healthy {
        service.Retries++
    }
}
```

Một method là function có receiver: phần parameter đặc biệt đứng trước tên method. `Summary` nhận `Service` value, nên receiver là một struct value được dùng để tạo kết quả. `Record` nhận `*Service`, vì hành vi này có ý định sửa pointee. Tên receiver không phải từ khóa và không phải một biến toàn cục bí mật; nó chỉ là parameter được viết ở vị trí riêng để gắn method với `Service`.

```
billing := Service{Name: "billing", Healthy: true}

fmt.Println(billing.Summary()) // billing
billing.Record(false)
fmt.Println(billing.Healthy, billing.Retries) // false 1
```

`billing.Record(false)` hợp lệ vì `billing` là variable addressable. Trong trường hợp này, Go cho phép cách viết ngắn của `(&billing).Record(false)`. Sự tiện lợi ấy không biến value receiver thành pointer receiver: method set của `Service` chỉ có methods khai báo với receiver `Service`; method set của `*Service` có cả methods receiver `Service` lẫn `*Service`.



Hình 14 — Method vẫn dựa trên receiver value. Summary dùng Service value; Record cần *Service để chọn pointee làm đích mutation. Mũi tên là trace semantics, không mô tả bộ nhớ vật lý.

Điều này giải thích một lỗi compiler rất có ích. Map index không addressable, nên Go không thể tự lấy địa chỉ ổn định cho một value trả về từ map để gọi pointer-receiver method:

```
// services["billing"].Record(false) // không hợp lệ

service := services["billing"]
service.Record(false)
services["billing"] = service
```

Đoạn sửa dùng đúng mô hình cũ: map lookup cho struct value, local variable `service` addressable, pointer-receiver method mutate local pointee, rồi caller gán struct value đã đổi về map. Nếu code thật cần nhiều nơi cùng giữ identity của một service, `map[string]*Service` có thể hợp lý; nhưng không dùng pointer chỉ để né phép gán lại mà chưa hiểu ownership.

Composition trước, embedding sau

Một service có endpoint và state; hai phần đó là các khái niệm riêng. Composition đơn giản nhất là đặt chúng vào những field có tên, để người đọc thấy quan hệ "có một":

```
type Endpoint struct {
    Host string
    Port int
}

type ProbeState struct {
    Healthy bool
    Retries int
}

type Service struct {
    Name      string
    Endpoint  Endpoint
    State     ProbeState
}
```

Service có Endpoint và ProbeState; không có quan hệ kế thừa nào được ngụ ý. Đây là cách composition thường rõ nhất khi domain có các phần độc lập. Embedding cũng là một công cụ của Go, nhưng nó đưa field và method được promote vào selector của type ngoài. Ta sẽ chỉ dùng nó khi một API có lợi từ sự promote đó; hiện tại field có tên giữ ownership và đường đi của data rõ hơn.

Interface xuất hiện ở nơi cần một hành vi

Một interface không phải bản thiết kế cha để mọi struct đi theo. Nó là một contract nhỏ: code tiêu dùng nói mình cần method nào, còn type nào có method set phù hợp thì dùng được. Với opspoke, phần in kết quả chưa cần toàn bộ Service; nó chỉ cần một dòng summary.

```

type SummarySource interface {
    Summary() string
}

func renderSummary(source SummarySource) string {
    return "target: " + source.Summary()
}

```

renderSummary không hỏi caller là Service, StaticTarget, hay một type tương lai. Nó chỉ gọi Summary. Không có từ khóa implements: assignment hoặc argument hợp lệ khi method set của type thỏa interface. Vì Summary có value receiver, cả Service và *Service đều có method đó trong method set.

```

type StaticTarget string

func (target StaticTarget) Summary() string {
    return string(target)
}

fmt.Println(renderSummary(Service{Name: "billing"}))
fmt.Println(renderSummary(StaticTarget("search (maintenance)")))

```

Kết quả là hai dòng có cùng format dù hai type không có ancestor chung. Interface ở đây được rút ra sau khi có consumer và hai provider có ý nghĩa. Nếu chỉ có một Service duy nhất và không có ranh giới cần thay thế, nhận trực tiếp Service thường dễ đọc hơn là thêm interface vì dự đoán tương lai.



Hình 15 — `renderSummary` chỉ cần `Summary() string`. Sơ đồ cho thấy dependency đi từ consumer tới contract nhỏ; `Service` và `StaticTarget` không cần khai báo một quan hệ kế thừa hay đăng ký implements.

Pointer receiver thay đổi interface satisfaction theo đúng rule method set. Nếu contract yêu cầu `Record(bool)`, `Service` value không thỏa vì method đó chỉ thuộc method set của `*Service`; `*Service` mới thỏa.

```

type ProbeRecorder interface {
    Record(bool)
}

func markFailed(recorder ProbeRecorder) {
    recorder.Record(false)
}

billing := Service{Name: "billing", Healthy: true}
markFailed(&billing) // *Service thỏa ProbeRecorder

// markFailed(billing) // sai: Service thiếu Record
  
```

Bài tập - chọn contract nhỏ: `renderSummary` cần biết `Name`, `Port`, `Healthy` và `Retries` hay chỉ cần `Summary() string`? Viết một type thứ hai có thể đi qua `renderSummary` mà không phải là `Service`, rồi giải thích vì sao interface đặt ở consumer thay vì được nhét sẵn vào mọi type.

Đáp án. Consumer chỉ cần `Summary() string`, nên interface nên giữ đúng một method đó. `StaticTarget` ở ví dụ trên là một provider thứ hai: nó có thể là dữ liệu đọc từ một report thay vì service sống. Đặt interface cạnh `renderSummary` khiến contract phản ánh nhu cầu thật của consumer; `Service` không phải biết trước mọi interface mà code khác sẽ cần.

Một API nhỏ, đủ để đọc bằng semantics

Ở cuối mạch này, `opsprobe` có thể diễn đạt ba ý định khác nhau mà không cần dựa vào danh từ mơ hồ như "reference type":

- `func (service Service) Summary() string` nhận snapshot receiver và tạo mô tả.
- `func (service *Service) Record(bool)` nhận pointer value, rồi sửa pointee có chủ ý.
- `func renderSummary(SummarySource) string` nhận interface value và chỉ phụ thuộc vào hành vi mà consumer cần.

Composition giữ data model có đường đi rõ ràng; methods đặt behavior cạnh type; interface tạo ranh giới khi có một consumer thực sự cần thay thế provider. Từ đây, chương về errors có thể cho những API này một cách báo thất bại rõ ràng: lỗi probe không nên chỉ đổi `Healthy` rồi biến mất, mà phải đi qua ranh giới function với thông tin đủ để caller quyết định.

Chương 4 — Biên lỗi: để caller quyết định

opsprobe vừa có thêm Service, state và một hàm ghi kết quả probe.

Phiên bản đầu của hàm trả bool: true nếu tên service tồn tại, false nếu không. Nó là một API hợp lệ khi caller chỉ cần biết có ghi được entry hay không.

Nhưng hãy đặt nó vào một tình huống vận hành. Màn hình báo billing không healthy. Có ít nhất hai câu hỏi khác nhau: service có được cấu hình không, hay service đã cấu hình nhưng probe tới endpoint thất bại? Một false không mang câu trả lời nào đi qua ranh giới function. Caller buộc phải đoán, hoặc đọc state đã bị đổi sau sự kiện.

Chương này không bắt đầu bằng một danh sách hàm trong package errors. Nó bắt đầu bằng quyền quyết định: nơi gọi cần nhận được failure nào để chọn thông báo, retry, hay dừng công việc; nơi phát hiện failure cần thêm context nào mà không phá mất identity của nguyên nhân.

Một failure chưa phải là một quyết định

Ta tách hành vi probe thành một function value. Nó chưa mở network; điều đó sẽ đến ở chương I/O. Hiện tại nó là một boundary có thể thành công hoặc trả về nguyên nhân thất bại:

```
type Endpoint struct {
    Host string
    Port int
}

type ProbeFunc func(Endpoint) error
```

ProbeFunc nhận endpoint value và trả về error. Nếu không có lỗi, result là nil. Nếu có lỗi, caller nhận một error value. error là interface của Go; điều đáng chú ý ở đây không phải interface đó có một method, mà là failure đi ra cùng result channel với một shape thống nhất. Không dùng panic để báo endpoint unreachable, và cũng không in log bên trong ProbeFunc rồi giả vờ thành công.

API ghi kết quả của `opsprobe` đổi từ câu hỏi “có entry không?” sang “tôi đã probe và cập nhật state được chưa?”

```
func applyProbe(  
    registry map[string]Service,  
    name string,  
    run ProbeFunc,  
) error
```

Signature nói một điều quan trọng trước khi ta đọc implementation: caller không nhận state mới hay một boolean mơ hồ. Nó nhận `nil` khi operation hoàn tất, hoặc một error value để quyết định nhánh kế tiếp. `registry` vẫn là map value dẫn tới shared map data; vì map đang lưu struct value, hàm vẫn lấy entry ra, sửa local `Service`, rồi gán lại entry như ở Chương 3.

Phân loại để quyết định, thêm context để điều tra

Không phải mọi string lỗi đều là contract. Một CLI có thể muốn đối xử riêng với “tên service không có trong config”, nhưng không nên parse câu tiếng Anh từ `err.Error()` để làm điều đó. Ta tạo một sentinel cho điều kiện mà caller có hành động riêng:

```
var ErrUnknownService = errors.New("service is not configured")
```

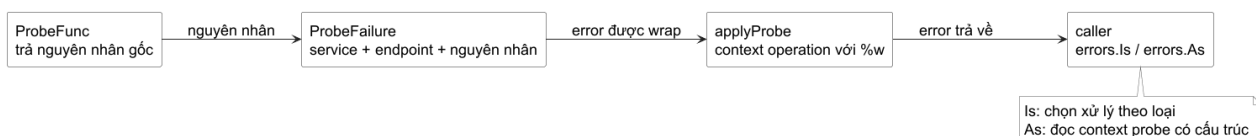
Khi tên không tồn tại, `applyProbe` không trả sentinel trần. Nó thêm thông tin operation và vẫn giữ nguyên identity bằng `%w`:

```
return fmt.Errorf("unknown %q: %w", name, ErrUnknownService)
```

`%w` không chỉ nối string. Nó tạo một error có thể unwrap về nguyên nhân. Vì vậy caller có thể kiểm tra ý nghĩa bằng `errors.Is` thay vì so câu chữ:

```
if errors.Is(err, ErrUnknownService) {
    fmt.Println("thêm service vào config trước khi probe")
    return
}
```

Trong câu lệnh trên, caller không cần biết hiện có bao nhiêu lớp context bên ngoài sentinel. `errors.Is` đi theo chuỗi wrapping. Nếu đổi `%w` thành `%v`, thông báo có thể nhìn giống nhau nhưng identity bị cắt; một điều kiện có thể phân loại được đã vô tình thành text chỉ dành cho người đọc.



Hình 16 — Biên lỗi không nuốt nguyên nhân. Mỗi lớp chỉ thêm context mà nó thực sự biết; caller chọn `errors.Is` khi cần một quyết định theo loại lỗi và `errors.As` khi cần dữ liệu từ một error cụ thể.

Một error có dữ liệu mà CLI thực sự cần

Sentinel trả lời “đây có phải loại lỗi đó không?”. Khi probe tới một endpoint đã có cấu hình nhưng thất bại, CLI cần thêm service nào và endpoint nào liên quan. Ta dùng custom error cho context có cấu trúc:

```

type ProbeFailure struct {
    Service  string
    Endpoint Endpoint
    Cause    error
}

func (failure *ProbeFailure) Error() string {
    return fmt.Sprintf(
        "probe %s at %s:%d: %v",
        failure.Service,
        failure.Endpoint.Host,
        failure.Endpoint.Port,
        failure.Cause,
    )
}

func (failure *ProbeFailure) Unwrap() error {
    return failure.Cause
}

```

Error tạo message cho người đọc. Unwrap nói rõ Cause vẫn là nguyên nhân nằm dưới lớp context này. ProbeFailure dùng pointer receiver; vì vậy khi ta cần lấy nó từ error chain, target của errors.As là *ProbeFailure:

```

var failure *ProbeFailure
if errors.As(err, &failure) {
    fmt.Printf("%s đang lỗi tại %s:%d\n",
        failure.Service,
        failure.Endpoint.Host,
        failure.Endpoint.Port,
    )
}

```

errors.As không hỏi “message có chứa host không?”. Nó tìm một error assignable vào type mà caller yêu cầu. Đây là lý do custom error nên có dữ liệu mà caller thật sự dùng, không phải chỉ là một struct bọc mỗi string.

Failure analysis: state nào phải được ghi, lỗi nào phải được trả?

Đây là implementation của boundary. Hãy đọc nó như một review: mỗi return mang thông tin nào, và state nào còn tồn tại sau return?

```
func applyProbe(
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error {
    service, found := registry[name]
    if !found {
        return fmt.Errorf("unknown %q: %w", name, ErrUnknownService)
    }

    if err := run(service.Endpoint); err != nil {
        service.Record(false)
        registry[name] = service
        failed := &ProbeFailure{
            Service:  service.Name,
            Endpoint: service.Endpoint,
            Cause:    err,
        }
        return fmt.Errorf("probe %q: %w", name, failed)
    }

    service.Record(true)
    registry[name] = service
    return nil
}
```

Nhánh thiếu config trả sớm: không có Service để mutate, nên registry không đổi. Nhánh probe thất bại vẫn ghi state `Healthy=false` và tăng retry, rồi trả failure có nguyên nhân. Đây không phải hai cơ chế báo lỗi cạnh tranh; state trả lời “lần quan sát gần nhất ra sao”, còn error trả lời “operation vừa gọi không hoàn tất vì sao”. Nhánh thành công ghi state rồi trả `nil`.

Khi kiểm thử code này, có ba contract cần chứng minh thay vì chỉ kiểm tra message:

- Với service thiếu, `errors.Is(err, ErrUnknownService)` là true và probe function không được gọi.
- Với probe thất bại, `errors.As` lấy được `*ProbeFailure`, `errors.Is` vẫn thấy nguyên nhân gốc, và state đã được ghi lại.
- Với probe thành công, error là `nil` và endpoint truyền vào probe đúng với config.

Lab `part4-error-boundaries` biến ba contract này thành test.

`ProbeFunc` được thay bằng function nhỏ trong test; không cần mở socket thật để chứng minh error boundary. Hãy mở `main_test.go` trước `main.go`, đổi riêng `%w` thành `%v` ở return cuối của nhánh probe lỗi, rồi chạy `TestApplyProbePreservesFailureCauseAndContext`. Message vẫn gần như cũ, nhưng test phải đổ vì `errors.Is` không còn thấy nguyên nhân. Khôi phục `%w`, sau đó tạm bỏ nhánh cancellation đặc biệt và chạy `TestApplyProbeCancellationDoesNotChangeHealthState`: lúc này feedback của test cho thấy cancellation đã bị viết nhầm thành health failure. Hai failure injection này đáng làm hơn là chép lại error chain đã có sẵn.

Bài tập - giữ lại điều gì? Một run trả `errors.New("connection refused")`. Nếu `applyProbe` đổi `%w` ở return cuối thành `%v`, message của CLI có thể vẫn hiển thị “connection refused”. Contract nào trong ba contract trên đã mất? Viết một assertion thể hiện điều đó.

Đáp án. Contract nhận biết nguyên nhân gốc mất: `errors.Is(err, errConnectionRefused)` sẽ false vì chuỗi error không còn unwrap tới error ban đầu. Context cho người đọc không đủ để thay thế identity cho code. Assertion cần tạo `errConnectionRefused`, gọi `applyProbe`, rồi yêu cầu `errors.Is(got, errConnectionRefused)` là true.

Điểm dừng của boundary này

Ta chưa retry, chưa timeout, và chưa cleanup tài nguyên; mọi thứ đó sẽ chỉ làm mạch này khó đọc nếu đưa vào trước. Hiện tại API đã có ranh giới rõ: `nil` nghĩa là operation hoàn tất, error giữ được nguyên nhân và context, còn caller dùng `Is` hoặc `As` đúng theo quyết định mình cần.

Phần tiếp theo của Chương 4 sẽ đặt error boundary này dưới áp lực của tài nguyên và cancellation: `defer` phải chạy ở đâu, `context` đi qua API nào, và vì sao `panic` không phải đường tắt thay cho một failure có thể dự đoán.

Cancellation không phải health signal

Một probe có thể bị hủy vì người dùng dừng CLI, vì request cha hết thời gian, hoặc vì service quản lý gọi đang shutdown. Không tình huống nào trong số đó chứng minh endpoint của `billing` đang down. Nếu `applyProbe` gộp mọi error vào nhánh `Record(false)`, một cancellation của caller sẽ tạo ra một health signal giả.

`context.Context` mang deadline và tín hiệu cancellation qua ranh giới API. Nó đi vào đầu signature, không được cất trong `Service`, và được truyền xuống function có thể chặn:

```
type ProbeFunc func(context.Context, Endpoint) error

func applyProbe(
    ctx context.Context,
    registry map[string]Service,
    name string,
    run ProbeFunc,
) error
```

Trước khi chạy probe, boundary kiểm tra cancellation để không bắt đầu I/O vô ích. Nếu cancellation đến trong lúc `run` đang làm việc, `run` phải tôn trọng `ctx` và trả `ctx.Err()`. `applyProbe` nhận diện hai nguyên nhân chuẩn `context.Canceled` và `context.DeadlineExceeded`, giữ chúng trong error chain, nhưng không thay health state:

```

if err := ctx.Err(); err != nil {
    return fmt.Errorf("probe canceled: %w", err)
}

if err := run(ctx, service.Endpoint); err != nil {
    if errors.Is(err, context.Canceled) ||
        errors.Is(err, context.DeadlineExceeded) {
        return fmt.Errorf("probe incomplete: %w", err)
    }

    service.Record(false)
    registry[name] = service
    failed := &ProbeFailure{
        Service: service.Name,
        Endpoint: service.Endpoint,
        Cause: err,
    }
    return fmt.Errorf("probe %q: %w", name, failed)
}

```

Đây là một policy của opspoke, không phải luật rằng cancellation không bao giờ có ý nghĩa vận hành. Tại boundary này, state `Healthy` có nghĩa là kết quả của lần quan sát hoàn tất. Một operation bị hủy chưa đưa ra quan sát đó. Caller vẫn có thể dùng `errors.Is(err, context.Canceled)` để dừng yên lặng, hoặc `errors.Is(err, context.DeadlineExceeded)` để báo timeout.

defer đứng cạnh acquisition

Cancellation không tự đóng mọi resource mà một function đã mở.

Ownership nằm ở nơi acquisition thành công: function nào có session thì function đó phải sắp cleanup trước khi gọi phần có thể return. Ta tách phần này khỏi health state để đọc được policy cleanup mà không lẫn nó với policy endpoint:

```

type ProbeSession interface {
    Close() error
}

type OpenSession func(Endpoint) (ProbeSession, error)
type SessionProbe func(
    context.Context,
    Endpoint,
    ProbeSession,
) error

func runWithSession(
    ctx context.Context,
    endpoint Endpoint,
    open OpenSession,
    run SessionProbe,
) (err error) {
    if err := ctx.Err(); err != nil {
        return err
    }

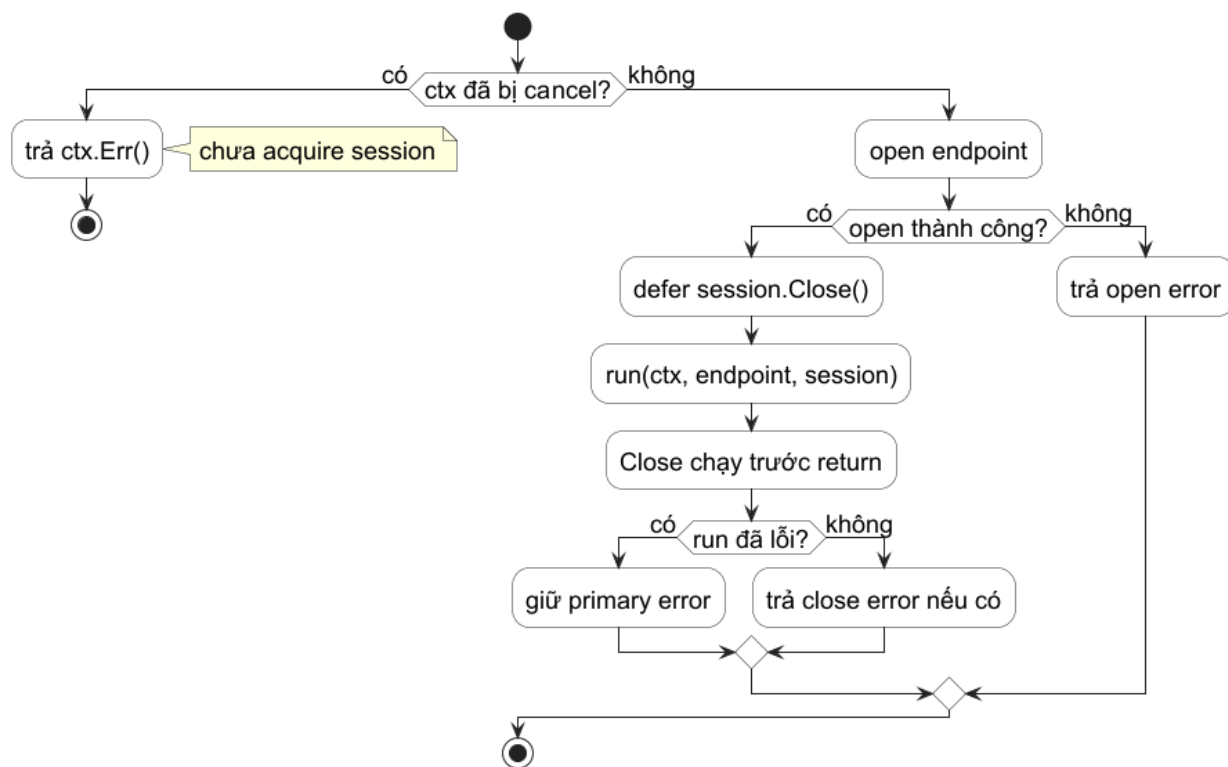
    session, err := open(endpoint)
    if err != nil {
        return fmt.Errorf("open session: %w", err)
    }
    defer func() {
        if closeErr := session.Close(); closeErr != nil {
            if err == nil {
                err = fmt.Errorf("close session: %w", closeErr)
            }
        }
    }()

    return run(ctx, endpoint, session)
}

```

defer được đăng ký ngay sau khi open thành công. Nếu open thất bại, chưa có session để đóng. Nếu run return theo bất kỳ nhánh nào, Close chạy trước khi runWithSession trả result cuối cùng. Named result err ở đây có lý do cụ thể: cleanup error chỉ trở thành result khi operation chính đã thành công. Nếu run đã có error, code giữ primary failure thay vì thay thế

nó bằng một lỗi close thứ cấp. Một hệ thống có yêu cầu lưu cả hai có thể dùng policy khác, nhưng phải làm điều đó có chủ ý.



Hình 17 — **defer** không phải một câu thần chú dọn dẹp toàn chương trình. Nó ràng buộc cleanup với scope đã nhận ownership. Error từ **Close** chỉ thay result khi **run** chưa có failure để bảo toàn nguyên nhân chính.

Lưu ý một ranh giới khác: caller tạo context dẫn xuất phải gọi **cancel**, kể cả khi deadline có thể tự hết. Trong code ngắn, pattern là `ctx, cancel := context.WithTimeout(parent, timeout)` ngay sau đó `defer cancel()`. `cancel` giải phóng resource liên quan tới context; còn `session.Close` giải phóng resource mà `runWithSession` đã acquire. Hai cleanup này có owner khác nhau.

Code review: đừng dùng **recover** để che failure dự đoán được

Một review có thể gặp đoạn sau và thấy nó “tránh crash”:

```
defer func() {  
    if recovered := recover(); recovered != nil {  
        err = fmt.Errorf("probe failed: %v", recovered)  
    }  
}()
```

Đoạn này không phải cách thay thế cho `return error`. Cancellation, timeout, config thiếu và connection refused đều là outcome mà API đã dự kiến; chúng phải đi qua error chain để caller phân loại được. Dùng `recover` ở đây biến bug lập trình như nil dereference hoặc invariant hỏng thành một string mơ hồ, trong khi state có thể đã bị thay đổi dở dang.

`panic` dành cho tình trạng không thể tiếp tục theo contract nội bộ hoặc cho những boundary được thiết kế riêng để cách ly code không tin cậy. `recover` chỉ có chỗ khi boundary đó có kế hoạch phục hồi state, ghi nhận đầy đủ và có contract rõ ràng sau recovery. `opsprobe` hiện không có boundary như vậy; để panic lộ ra trong test thường trung thực hơn là giả nó thành “probe failed”.

Bài tập - trace cleanup: `ctx` bị cancel sau khi open trả một session, còn `run` trả `context.Canceled`. `Close` có được gọi không? Error cuối giữ identity nào? `registry["billing"]` có nên đổi `Healthy` hay `Retries` không?

Đáp án. `Close` được gọi đúng một lần vì `defer` đã được đăng ký sau acquisition. `RunWithSession` giữ `context.Canceled` là primary error nếu `Close` cũng lỗi; `applyProbe` bọc nguyên nhân đó để `errors.Is(err, context.Canceled)` vẫn true. Vì operation không hoàn tất một quan sát endpoint, policy hiện tại giữ nguyên `Healthy` và `Retries`.

Kết chương: failure có đường đi

Một error boundary tốt không cố làm mọi failure giống nhau.

`ErrUnknownService` cho caller một loại lỗi để cấu hình; `ProbeFailure` giữ context của quan sát thật; wrapping giữ identity; cancellation đi qua `Context` nhưng không bị viết nhầm thành health signal; `defer` đóng resource mà scope đã nhận ownership. `panic` vẫn là tín hiệu khác, không phải lỗi tắt để khởi thiết kế result.

`opsprobe` nay nói rõ điều nó sở hữu, quan sát và để caller quyết định. Tiếp theo: package boundary, export và dependency kín.

Bốn câu hỏi trước khi merge một error boundary

- Caller có một quyết định khác nhau cho từng outcome không? Nếu có, hãy giữ identity bằng sentinel hoặc type có dữ liệu; đừng bắt caller parse message.
- Failure này có phải quan sát về domain không? Timeout hay cancellation của caller không tự động là bằng chứng endpoint unhealthy.
- Resource được acquire ở đâu, và cleanup có được đăng ký ngay sau acquisition không? Mỗi `defer` nên chỉ rõ scope nào đang nhận ownership.
- Nếu cleanup cũng lỗi, primary failure nào không được phép biến mất? Policy có thể khác giữa hệ thống, nhưng cần được kiểm thử như một contract.

Khi bốn câu trả lời hiện ngay trong signature, error chain, state transition và test, người đọc sau không cần đoán error có bị nuốt, retry có bị tăng sai, hay resource có bị bỏ quên. Đó là chuẩn bị cần thiết trước khi tách code thành package.

Chương 5 — Package là ranh giới

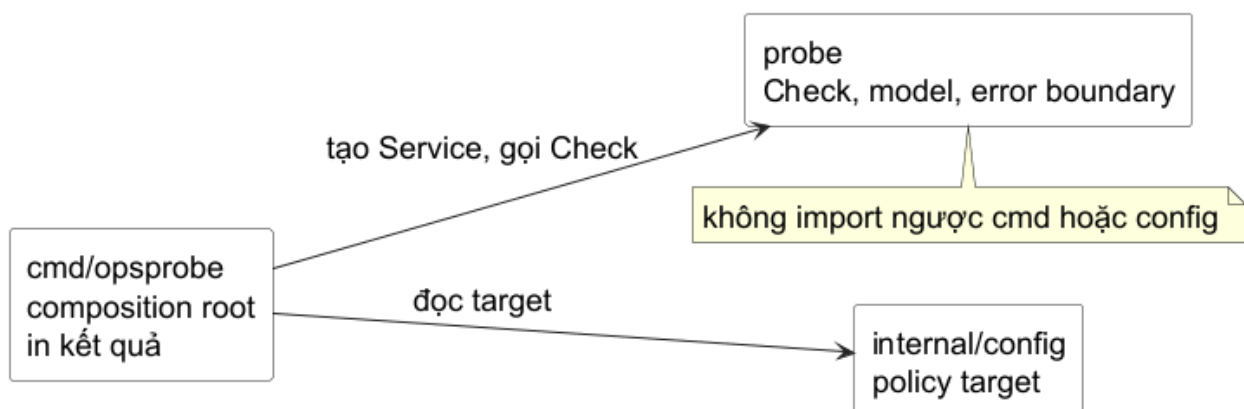
Sau Chương 4, `opsprobe` có một API cho một lần probe. Nhưng lab vẫn là một package `main` chứa `model`, `policy error`, `config` giả và cách in ra `terminal`. Khi mọi thứ còn ngắn, một tệp là lựa chọn tốt. Vấn đề xuất hiện không phải vì số tệp tăng, mà vì một thay đổi ở cách in CLI có thể nhìn thấy mọi chi tiết của probe, và một thay đổi config có thể kéo domain model đi cùng.

Ta sẽ không mở đầu bằng cây thư mục “chuẩn”. Ta bắt đầu bằng một review: code nào phải biết `fmt.Println`, code nào phải biết `endpoint`, và code nào có quyền quyết định config được đọc từ đâu? Khi câu trả lời khác nhau, `boundary` cũng phải khác nhau.

Một dependency graph có thể đọc từ ngoài vào trong

Refactor đầu tiên của `opsprobe` có ba vai:

- `cmd/opsprobe` là `command`: nhận các dependency, gọi use case và trình bày kết quả.
- `probe` là `package importable`: sở hữu model cho một lần kiểm tra và không in ra `terminal`.
- `internal/config` là `implementation detail` của application: biết target mặc định hôm nay, nhưng không trở thành API công khai mà module khác được hứa hỗ trợ.



Hình 18 — Dependency direction là công cụ review. `cmd/opsprobe` là composition root; nó biết các package cụ thể. `probe` không biết CLI hay config source nên có thể được đọc, test và thay thế runner mà không kéo presentation theo.

`go.mod` đặt module path, còn mỗi directory Go là một package. Vì lab dùng module `example.com/golang-master/part5-package-design`, import path của package probe là `example.com/golang-master/part5-package-design/probe`. Một package có thể gồm nhiều file trong cùng directory; chia file không tạo thêm boundary nếu chúng vẫn khai báo cùng package probe.

```
part5-package-design/
├── go.mod
├── cmd/opsprobe/main.go
├── probe/doc.go
├── probe/check.go
└── internal/config/config.go
```

Tên `cmd` là convention hữu ích khi repository có thể chứa nhiều executable; bản thân compiler không trao ý nghĩa đặc biệt cho nó. `main.go` cũng chỉ là tên file theo convention. Boundary thực sự đến từ package clause, import path và hướng dependency.

Export là lời hứa, không phải cách sửa compile error

Trong probe, caller cần tạo service, truyền một runner và đọc result. Những khái niệm ấy mới cần export:

```
package probe

type Endpoint struct { Host string; Port int }
type Service struct { Name string; Endpoint Endpoint }

type Runner func(context.Context, Endpoint) error
type Result struct { Service string }

func Check(
    ctx context.Context,
    service Service,
    run Runner,
) (Result, error)
```

Chữ cái đầu viết hoa không làm type “quan trọng hơn”; nó mở một contract cho package khác. Check được export vì command thực sự gọi nó. Helper thêm context error ở bên trong có thể giữ unexported vì chưa có caller nào ngoài probe cần nó. Export trước rồi hy vọng một consumer xuất hiện là cách tạo API phải mang gánh nặng tương thích không cần thiết.

Package documentation cũng là một phần của contract. File `probe/doc.go` bắt đầu bằng comment `Package probe . . .`, nói package chịu trách nhiệm gì và không chịu trách nhiệm gì. Comment của export nên mô tả behavior mà caller có thể dựa vào, không chỉ lặp lại tên type.

`internal` là boundary do compiler kiểm tra

Target mặc định là policy của application hiện tại. Ta để nó trong `internal/config`:

```
package config

type Target struct {
    Name string
    Host string
    Port int
}

func DefaultTargets() []Target {
    return []Target{
        {Name: "billing", Host: "billing.internal", Port: 8443},
    }
}
```

Package ở `internal` không phải “private bằng thỏa thuận”. Go chỉ cho code nằm trong cây thư mục của parent import nó. Vì `cmd/opsprobe` nằm dưới module root - parent của `internal` - nó import được `internal/config`. Một module khác import `example.com/golang-master/part5-package-design/internal/config` sẽ bị go command từ chối.

Đây không phải lý do để nhét mọi package vào `internal` theo phản xạ. Với binary tự chứa, đó thường là default tốt vì không hứa API ngoài. Với probe

trong lab này, ta cố tình để nó importable để boundary giữa domain operation và CLI được nhìn thấy. Nếu mai kia không còn consumer ngoài application, nó cũng có thể trở thành internal; vị trí phải theo consumer thật, không theo cây thư mục đẹp mắt.

Composition root chuyển dữ liệu, không giấu dependency

`config.Target` và `probe.Service` không phải cùng type. Chúng thuộc hai boundary khác nhau: `config` nói dữ liệu được khai báo thế nào; `probe` nói một operation cần gì. `cmd/opsprobe` là nơi phù hợp để map giữa chúng:

```
for _, target := range config.DefaultTargets() {
    service := probe.Service{
        Name: target.Name,
        Endpoint: probe.Endpoint{
            Host: target.Host,
            Port: target.Port,
        },
    },
}

result, err := probe.Check(ctx, service, run)
if err != nil {
    fmt.Printf("%s: %v\n", service.Name, err)
    continue
}
fmt.Printf("%s healthy=true\n", result.Service)
}
```

Mapping này trông như boilerplate nhỏ, nhưng nó giữ dependency direction trung thực: `probe` không cần import `config`, `config` không cần biết `Result`, và code in terminal không rò vào domain package. Khi source `config` chuyển từ default sang file hay environment ở chương sau, `probe.Check` không đổi chỉ vì đường đi của `config` đã đổi.

Một refactor có kiểm chứng, không phải một lần dời file

Lab kiểm tra ba điều có thể bị hỏng khi tách package:

- `probe.Check` chuyển cancellation thành error có thể nhận diện bằng `errors.Is`, và không chạy runner khi context đã bị cancel.
- Result thành công mang đúng service name.
- `internal/config` trả dữ liệu độc lập theo từng lần gọi; caller sửa slice nhận được không làm thay default của lần sau.

Test cuối không phải để khẳng định “mọi slice đều dangerous”. Nó chứng minh ownership của config factory: `DefaultTargets` tạo dữ liệu mới cho caller thay vì phát một slice dùng chung. Đây là cùng câu hỏi value semantics của Chương 2, nay được đặt vào một package boundary.

Lần này không bắt đầu từ implementation đã tách. Mở `labs/part5-package-refactor` trong VS Code: điểm xuất phát là một `main.go` monolithic đang chạy. Chạy `go run` để giữ behavior làm mốc, rồi chạy `go test -tags exercise ./...`. Lỗi compiler vì package `probe` chưa tồn tại là tín hiệu bắt đầu, không phải lỗi để lờ đi: test đang đòi một public boundary mà code chưa có. Từ requirement và test, anh tự quyết định file nào đi vào `probe`, file nào là `internal/config`, và command map hai model ở đâu. Khi xong, `go run ./cmd/opsprobe` phải giữ output, còn `go test -tags exercise ./...` và `go vet ./...` phải xanh.

Đáp án — chỉ đọc sau khi đã tự làm. Reference implementation nằm ở `labs/part5-package-design`, nhưng chỉ nên mở sau khi acceptance test đã cho phản hồi đầu tiên. Ranh giới cần giữ là: `probe` export operation mà test gọi, `internal/config` giữ policy default và trả slice mới, còn `cmd/opsprobe` là nơi map data rồi in kết quả. Không có lý do để `probe` import ngược config hoặc terminal.

Bài tập - đặt code ở đâu? Một bạn muốn thêm `fmt.Println` vào `probe.Check` để in progress, và muốn `probe` gọi `config.DefaultTargets` để đỡ mapping ở `main`. Với dependency graph trên, hai thay đổi đó tạo ra rủi ro gì? Đề xuất nơi thay thế cho mỗi việc.

Đáp án. `fmt.Println` kéo `presentation` vào `package` có thể `import`; `command` nên quyết định `format` và nơi ghi `output`. Để `probe` gọi `config` đảo `dependency` từ `domain operation` sang `application policy`, khiến `test probe` phải mang `config` theo. `Progress` có thể là `result/event` do `caller` trình bày; `mapping` nên ở `command` hoặc một `application service` nằm phía ngoài cả `probe` lẫn `config`.

Một config source là dữ liệu không tin cậy, không phải endpoint

Requirement tiếp theo đến từ người vận hành: trong một lần chạy, họ muốn đổi `endpoint` mặc định mà không sửa `source`. Ta chọn một biến môi trường có shape rõ ràng: `OPS_PROBE_TARGET=payments.internal:9443`. Đây là `capability` của `application`, không phải `capability` của `probe`.

`probe.Check` chỉ nhận `Endpoint` đã hợp lệ; nó không có lý do để biết chuỗi đó đến từ môi trường, file hay một flag tương lai.

Điều này giữ một ranh giới quan trọng. `Environment` luôn đưa vào `string`; `endpoint` cần `host` và `port` có ý nghĩa. Việc tách, chuyển kiểu và kiểm tra `range` là công việc của `internal/config`. `Command` đưa `implementation` thật `os.LookupEnv` vào `boundary` ấy, rồi chỉ nhận `[]Target` hoặc `error`. Shape của `API` đủ nhỏ để `test` mà không đổi `environment` của máy:

```
type LookupEnv func(string) (string, bool)

func LoadTargets(lookup LookupEnv) ([]Target, error)
```

Đây không phải một `dependency-injection framework`. `LookupEnv` chỉ là `function type` nói đúng `dependency` mà `config` cần: với một `key`, trả `value` và cho biết `key` có tồn tại hay không. Trong `test`, một `closure` nhỏ thay `os.LookupEnv`; trong `command`, `os.LookupEnv` thỏa đúng `signature`. Nhờ vậy `test` mô tả `input` rõ ràng mà không có `test` nào phải sửa `environment process` chung.

`host:port` cũng không nên được cắt bằng `strings.Split`. Dấu : xuất hiện trong `IPv6`, nên `net.SplitHostPort` là `parser` phù hợp cho `grammar socket`. `payments.internal:9443` là `input` hợp lệ; `IPv6` cần `bracket` như

[2001:db8::9]:9443. Sau khi tách, port vẫn là text: `strconv.Atoi` chuyển nó thành `int`, rồi config kiểm tra range `1..65535` trước khi tạo target. Mọi input lỗi trả error có context để CLI in ra và dừng với exit code khác 0.

Ở đây không cần sentinel mới. Caller hiện chỉ có một policy cho config lỗi: báo người dùng và dừng trước khi probe bắt đầu. Nếu sau này CLI cần phân biệt lỗi syntax với secret bị thiếu để đưa remediation khác nhau, đó mới là lúc error identity đáng được đưa vào contract. Chương 4 không dạy “mọi error phải có type”; nó dạy chỉ giữ identity khi caller có quyết định khác nhau.

Thực hành - requirement thay đổi sau refactor. Sau khi hoàn thành lần tách package đầu tiên trong `labs/part5-package-refactor`, chạy `go test -tags configexercise ./....`. Test đòi `internal/config.LoadTargets` nhận một function đọc environment và đổi endpoint khi có `OPS_PROBE_TARGET`. Tự chọn file, API và error message; constraints là dùng `net.SplitHostPort`, từ chối host rỗng và port ngoài `1..65535`, còn `cmd/opsprobe` mới được gọi `os.LookupEnv`. Khi test xanh, chạy command với `OPS_PROBE_TARGET=payments.internal:9443` để kiểm tra đường đi runtime.

Đáp án — chỉ đọc sau khi đã tự làm. Reference implementation giữ `LoadTargets` trong `internal/config`, bắt đầu từ `DefaultTargets` để ownership của slice không đổi, rồi chỉ override host và port của target đầu. Command xử lý error ở process boundary bằng `stderr` và exit code 2; probe không thay đổi. Đây là dấu hiệu refactor tốt: requirement về source config đi qua một package, trong khi operation kiểm tra endpoint không cần biết nguồn dữ liệu vừa đổi.

Code review: một field không có trạng thái thứ hai

Sau config change, ta đọc lại public API thay vì vội thêm feature khác. Check có hai outcome: return error khi operation không hoàn tất, hoặc return Result khi runner thành công. Trong contract này, Result.Healthy luôn là true. Field đó trông vô hại, nhưng nó gợi cho caller một trạng thái thứ hai mà function không bao giờ trả về.

Đây là một bug thiết kế, không phải bug compiler. Một caller mới có thể viết `if !result.Healthy { retry() }`, rồi tin rằng branch ấy có ý nghĩa. Thực tế, failure đã đi qua error; result zero value đi cùng error không phải một observation “unhealthy”. Giữ bool chỉ vì câu in terminal cần chữ true làm biên API rộng hơn dữ liệu mà operation thực sự cung cấp.

Ta thu hẹp Result còn service đã hoàn tất. cmd/opsprobe đã có nhánh `err == nil`, nên chính command - nơi sở hữu presentation - in `healthy=true`. Không có information bị mất: success đã là bằng chứng của true; failure vẫn là error với context. Đây là một lần refactor đặc biệt đáng làm vì test không đo trước khi đổi. Test cũ xanh, nhưng contract cũ làm người đọc có thể suy luận sai.

```
// Result identifies the service whose check completed successfully.
type Result struct {
    Service string
}
```

Một comment tốt giờ cũng có thể viết chính xác hơn: Result không “chứa health status”, mà định danh service có check thành công. Khi API thật sự cần diễn đạt nhiều trạng thái quan sát - ví dụ healthy, degraded, unreachable với timestamp và latency - đó sẽ là một model khác, được thiết kế cùng requirement và test riêng. Đừng giữ placeholder cho một feature chưa tồn tại chỉ để API trông linh hoạt.

Thực hành - review dưới ràng buộc. Trong `labs/part5-package-design`, tạm giữ `Healthy bool` trong `Result` rồi viết ra hai outcome mà `Check` có thể trả. Sau đó bỏ field, sửa test và chỉ sửa presentation ở command cho đến khi `go test ./...` cùng `go run ./cmd/opsprobe` đều xanh. Không được chuyển failure thành `Result{Healthy: false}`: điều đó làm mất error chain mà Chương 4 đã xây. Trong bản refactor độc lập, `public_boundary_test.go` là contract tối thiểu; hãy sửa implementation theo test, không thêm field chỉ để đoán tương lai.

Đáp án — chỉ đọc sau khi đã tự làm. `Result{Service: service.Name}` là đủ cho nhánh thành công. Sau `err == nil`, command tự in literal `healthy=true`. Nếu một ngày cần health state mà vẫn giữ operation failure riêng, API sẽ cần một contract mới nói rõ result nào có thể xuất hiện cùng error nào; đó không phải việc của bool hiện tại.

Điểm dừng: package đủ nhỏ để thay đổi

Chưa có một “architecture hoàn chỉnh” nào ở đây: chưa parse file, chưa dùng generic, chưa tạo package cho từng noun. Ta chỉ có một graph mà mỗi mũi tên trả lời được “vì sao package này cần biết package kia?”. Đó là mức cấu trúc đủ để tiếp tục thay đổi code mà không biến mọi thay đổi thành sửa main.

Chương 6 — Thay đổi không sợ hãi

Ở cuối Chương 5, `opsprobe` đã biết lấy một endpoint từ config và gọi `probe.Check`. Nhưng một yêu cầu rất bình thường vẫn khó chứng minh: khi operator đặt `OPS_PROBE_TARGET=payments.internal:9443`, endpoint nào thực sự đi tới runner? Chạy CLI và nhìn dòng `billing healthy=true` không trả lời được. Dòng đó không in host hay port; kể cả có in, một lần chạy tay vẫn không phải contract mà code giữ được khi nó đổi tiếp.

Đây là lúc nhiều codebase đi sai hướng. Có người viết test gọi `main`, sửa environment process chung, chụp stdout, rồi hy vọng không đụng `os.Exit`. Có người khác bỏ test vì “CLI chỉ vài dòng”. Cả hai đều né câu hỏi thiết kế: phần nào là orchestration có thể kiểm chứng, phần nào là process boundary chỉ nên mỏng và rõ?

Một smoke test không biết điều ta cần biết

`main` hiện làm bốn việc cùng lúc: lấy environment thật, gọi config, gọi `probe`, quyết định exit code và in terminal. Một test gọi nó chỉ cho biết process không chết ở đường happy path. Nó không thể đưa một environment giả vào an toàn, không thể quan sát endpoint runner nhận được, và không thể tách config error khỏi probe error mà không đụng side effect của process.

Vấn đề không phải `main` là xấu. `os.LookupEnv`, `stdout`, `stderr` và `os.Exit` đều có chỗ đúng: chúng là ranh giới nơi chương trình gặp process. Vấn đề là logic cần quyết định trước khi chạm ranh giới ấy chưa có một đơn vị để test gọi.

Ta tạo package nội bộ `internal/app`. Nó không thay thế `probe` hay `config`; nó phối hợp hai package đó. Run nhận đúng hai dependency biến đổi được trong test: function đọc environment và runner thực hiện `probe`. Nó trả outcome thay vì in:

```

type Outcome struct {
    Service string
    Err      error
}

func Run(
    ctx context.Context,
    lookup config.LookupEnv,
    runner probe.Runner,
) ([]Outcome, error)

```

`Outcome.Err` là failure của một target đã được cấu hình; CLI có thể in nó rồi tiếp tục target khác. `error` return riêng là failure không tạo được danh sách outcome có nghĩa, như config không hợp lệ hoặc cancellation của toàn bộ run. Việc tách hai đường này không phải để tạo type cho đẹp: caller có policy khác nhau. Chương 4 từng đặt câu hỏi “caller cần quyết định gì?”; ở đây câu trả lời quyết định shape của result.

Refactor đến khi test có thứ để nắm

Run load target, map `config.Target` sang `probe.Service`, gọi `probe.Check`, rồi giữ mỗi kết quả trong một `Outcome`. Nó không đọc `os.LookupEnv`, không gọi `fmt.Printf` và không được gọi `os.Exit`. Những side effect ấy còn ở `cmd/opsprobe/main.go`:

```

outcomes, err := app.Run(
    context.Background(),
    os.LookupEnv,
    runner,
)
if err != nil {
    fmt.Fprintln(os.Stderr, err)
    os.Exit(2)
}

for _, outcome := range outcomes {
    if outcome.Err != nil {
        fmt.Printf("%s: %v\n", outcome.Service, outcome.Err)
        continue
    }
    fmt.Printf("%s healthy=true\n", outcome.Service)
}

```

Đoạn main vẫn có logic tối thiểu: policy trình bày và process exit nằm lộ ra, dễ review. Nhưng test không cần gọi nó. Test nằm cạnh Run, đưa một closure làm LookupEnv và một probe. Runner ghi lại endpoint được gọi. Không có socket, environment toàn cục hay terminal nào bị đụng tới.

Trước khi viết assertion, hãy tách ba thứ bằng mắt. Chúng ngăn test biến thành bản sao của implementation:

Test kiểm soát	Test quan sát	Test không cần chạm
LookupEnv trả target nào; runner thành công hay lỗi	endpoint runner nhận; Outcome hoặc error trả về	environment process, stdout, stderr, os.Exit

Bảng 3 — Phạm vi của unit test quanh Run

Nếu một requirement chỉ hiện ra ở cột cuối, Run chưa phải đơn vị đúng để test; đó là việc của integration test ở một thời điểm có lý do. Nếu nó nằm ở cột đầu và cột giữa, unit test có thể cho feedback nhanh mà không giả lập cả process.

```

func TestRunMapsEnvironmentTargetToRunner(t *testing.T) {
    var got probe.Endpoint

    outcomes, err := Run(context.Background(), lookup(
        "OPS_PROBE_TARGET", "payments.internal:9443",
    ), func(_ context.Context, endpoint probe.Endpoint) error {
        got = endpoint
        return nil
    })

    if err != nil {
        t.Fatalf("Run() error = %v", err)
    }
    if got != (probe.Endpoint{Host: "payments.internal", Port: 9443}) {
        t.Fatalf("runner endpoint = %+v", got)
    }
    if outcomes[0].Err != nil || outcomes[0].Service != "billing" {
        t.Fatalf("outcome = %+v", outcomes[0])
    }
}

```

Đây là unit test không vì nó ngắn, mà vì nó kiểm chứng một đơn vị có boundary hẹp. Test hỏi một điều quan sát được: input config có đi tới runner đúng không, và result có được gán với service đúng không. Nó không kiểm tra implementation detail như Run dùng bao nhiêu local variable hay loop viết thế nào.

Hai error, hai policy

Thử thêm hai test trước khi đọc hết implementation. Với `OPS_PROBE_TARGET=payments.internal:0`, config phải error trước khi runner chạy. Đây là invalid input của cả run, nên Run return error và không có outcome nữa vời. Với runner trả `errors.New("connection refused")`, target đã tồn tại và operation đã bắt đầu; Run giữ `Outcome{Service: "billing", Err: ...}` để command có thể báo một dòng lỗi rồi tiếp tục khi danh sách target sau này dài hơn.

Cancellation khác với cả hai. `probe.Check` giữ identity `context.Canceled`, còn Run dừng thay vì biến cancellation của toàn bộ

run thành một outcome per-service. Từ bên ngoài, ba policy hiện rõ:

Tình huống	Run trả gì	Command nên làm gì?
Config không hợp lệ	error	In stderr, exit 2, không gọi runner.
Một probe thất bại	Outcome.Err	Báo service đó; policy hiện tại cho phép đi tiếp.
Context bị cancel	error giữ identity	Dừng run; không ghi health signal giả.

Bảng 4 — Policy cho lỗi cấu hình, lỗi probe và cancellation

Không phải test nào cũng cần fake. Config parser đã có test input/output riêng ở Chương 5; probe.Check đã có test cancellation và wrapping. Run chỉ test phần nối ba contract ấy. Nếu một test chỉ lặp lại assertion của package dưới nó, nó làm refactor khó hơn mà không tăng niềm tin.

Thực hành - remove and rebuild. Mở `labs/part6-testable-command`, đọc `internal/app/run_test.go` trước. Đừng chạy command trước; hãy dự đoán mỗi test đang bảo vệ policy nào. Sau đó tạm đổi tên Run để compiler báo các điểm mà test đang đòi contract, rồi tự dựng lại function từ test và các type đã có. Khi unit test xanh, chạy `go run ./cmd/opsprobe`. Cuối cùng đổi runner trong `main` thành một runner trả error và xác minh CLI in service name thay vì nuốt failure. Khôi phục runner thành công trước khi rời lab.

Đáp án — chỉ đọc sau khi đã tự làm. Run cần gọi `config.LoadTargets` đúng một lần trước loop, map từng `Target` sang `probe.Service`, rồi append outcome thành công hoặc failure của từng probe. Chỉ config error và cancellation thoát khỏi Run qua `return error`. `main` giữ environment thật, output và exit code. Lời giải không phải “dùng mock framework”; hai function value đang có đã là seams vừa đủ.

Khi một test case che mất các policy khác

Test đầu tiên của Run cố ý chỉ dùng một override hợp lệ. Nó trả lời câu hỏi của refactor: endpoint có thật sự đi tới runner không? Nhưng nếu cứ nhân bản kiểu test đó cho từng input xấu, test suite sẽ dần thành một hành lang dài của những hàm gần giống nhau. Lúc ấy người đọc không còn nhìn thấy các rule của config; họ chỉ thấy rất nhiều lần gọi LoadTargets.

Một reviewer tốt sẽ hỏi khác đi: parser này có bao nhiêu *policy* độc lập? Với OPS_PROBE_TARGET, ta cần giữ ít nhất hai. Variable vắng mặt dùng default đã công bố. Variable có mặt phải là một host:port hợp lệ, có host và port trong khoảng TCP. Mỗi hàng test là một ví dụ cho một policy, không phải một input ngẫu nhiên để tăng coverage.

Đây là lúc bảng test có ích. Không phải vì Go community có một nghi thức tên là “table-driven test”, mà vì bảng làm lộ phần thay đổi và phần giữ nguyên. raw, found và want thay đổi theo case; cách gọi parser và assertion về một target duy nhất được giữ chung:

```
for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        targets, err := LoadTargets(
            func(string) (string, bool) {
                return tt.raw, tt.found
            },
        )
        if err != nil {
            t.Fatalf("LoadTargets() error = %v", err)
        }
        if len(targets) != 1 || targets[0] != tt.want {
            t.Fatalf("LoadTargets() = %+v", targets)
        }
    })
}
```

t.Run không chỉ để kết quả trông có tổ chức. Khi một case vỡ, go test -v sẽ nói case nào vỡ, thay vì để anh đếm hàng trong slice. Vì vậy name phải mô tả behavior: “port zero” tốt hơn “case 4”, còn “uses the documented default” nhắc cả người bảo trì rằng default là một phần contract.

Review một bảng test trước khi tin nó

Thêm các input bị từ chối vào một table thứ hai trong `internal/config/config_test.go`:

```
tests := []struct {
    name string
    raw  string
}{
    {name: "missing port", raw: "payments.internal"},
    {name: "empty host", raw: ":8443"},
    {name: "non-numeric port", raw: "payments.internal:https"},
    {name: "port zero", raw: "payments.internal:0"},
    {name: "port above TCP range", raw: "payments.internal:65536"},
}
```

Điểm dễ bỏ sót không nằm ở vòng `for`, mà ở `assertion`. Test này chỉ hứa rằng từng input không hợp lệ bị từ chối; nó không hứa nguyên văn `error message`. Message là chi tiết có thể sửa để tốt hơn cho operator. Rule “input này không được phép tạo target” mới là contract. Nếu CLI hoặc API cần phân loại lỗi bằng máy, khi đó mới có lý do để thiết kế `error type` hay `sentinel` rõ ràng; đừng dùng `string comparison` như một cách thay thế cho thiết kế đó.

Có một bẫy nhỏ khi viết `subtest`: đừng lấy địa chỉ của biến `loop` rồi giữ nó để dùng sau vòng lặp. Ở đây `t.Run` chạy đồng bộ, closure đọc `tt` ngay trong `body`, nên không có goroutine nào sống qua iteration. Bài về `parallel subtest` sẽ quay lại bẫy này cùng `race detector`; hiện tại ta giữ test tuân tự để `policy parser` là thứ duy nhất cần nghĩ.

Bài review ngắn. Trước khi chạy test, hãy tự phân loại năm hàng trên: hàng nào bị `net.SplitHostPort` từ chối, hàng nào đi qua parser nhưng bị `validation` của sách từ chối? Sau đó thêm một case hợp lệ cho IPv6 theo dạng `[2001:db8::10]:443`. `Assertion` của nó phải kiểm tra `Host` không còn dấu ngoặc và `Port == 443`, thay vì chỉ kiểm tra `err == nil`.

Đáp án — chỉ đọc sau khi đã tự làm. Thiếu port và :8443 không tạo được target hợp lệ ở bước parse/host check. https, 0 và 65536 đi xa hơn: chúng cần conversion hoặc range validation. Với IPv6, `net.SplitHostPort` tách dạng có ngoặc thành host `2001:db8::10` và port `443`; chính vì vậy test nên ghi điều đó thành contract. Khi bảng bắt đầu chứa các cột không cùng một rule, hoặc mỗi row cần setup rất khác, tách test ra sẽ rõ hơn là nhồi thêm cột.

Chạy đúng bằng chứng

Khi suite xanh, `go test ./...` trả lời câu hỏi có giá trị nhưng rộng: có package nào vi phạm contract không? Lúc điều tra một report cụ thể, chạy tất cả test nhiều lần lại làm mất ngủ cảnh. Nếu operator báo rằng port 0 từng lọt qua validation, ta cần nhìn đúng rule đó trước, rồi mới mở rộng phạm vi.

Từ thư mục `labs/part6-testable-command`, lệnh sau chọn một subtest bằng đường dẫn tên của nó:

```
$case = 'TestLoadTargetsRejectsMalformedOverrides/port_zero'
go test -run $case -v ./internal/config
```

Đây không phải shortcut để né test đầy đủ trước khi commit. Nó là một kính lúp trong lúc debug: output giữ tên parent test và case `port_zero`, nên anh biết mình đang xem policy nào. Sau khi sửa bug, chạy lại package rồi chạy toàn bộ suite; một lỗi parser có thể làm app thất bại trước cả khi runner được gọi.

Hãy thử thay đổi tạm thời điều kiện range trong `LoadTargets` để port 0 đi qua, chỉ để xem subtest vỡ như thế nào. Đừng commit thay đổi đó. Bài học không phải thuộc câu lệnh `-run`; nó là phân biệt vòng lặp điều tra ngắn với bằng chứng đủ rộng để nhận một thay đổi vào codebase.

Từ đây Chương 6 đã có hai loại test khác nhau: RUN kiểm chứng việc nối các boundary, còn config subtest giữ một ma trận input nhỏ nhưng có ý nghĩa. Phần tiếp theo sẽ tạo một failure mà unit test xanh vẫn chưa bắt được - hai goroutine cùng chạm vào một vùng dữ liệu.

Một test xanh vẫn có thể bỏ sót race

opsprobe hiện chưa chạy probe song song; đó là lựa chọn có chủ đích khi ta còn đang học boundary và policy lỗi. Vì vậy sẽ là sai nếu em giả vờ vừa phát hiện một race trong code của nó. Thay vào đó, hãy tách đúng failure cần học: nhiều worker cùng ghi vào một counter hoàn thành. Đây là loại bug có thể trông vô hại khi chạy một lần, vì không có compiler error và unit test tuần tự vẫn xanh.

Data race xảy ra khi nhiều goroutine truy cập cùng một biến đồng thời, có ít nhất một lần ghi, và không có synchronization phù hợp. Đây là định nghĩa về hành vi, không phải một lời phán đoán dựa trên việc count cuối cùng “có vẻ đúng”. `successes++` không phải một hành động nguyên tử ở cấp source; nó đọc giá trị cũ, tạo giá trị mới rồi ghi lại. Hai goroutine có thể xen vào giữa các bước ấy.

Lab `labs/part6-race-detector/broken` giữ một counter cố ý sai. Test tạo 128 goroutine, mỗi goroutine gọi đúng một lần `RecordSuccess`:

```
type Counter struct {
    successes int
}

func (c *Counter) RecordSuccess() {
    c.successes++
}
```

Chạy test thường có thể không nói gì đáng ngờ. Để đưa memory access vào quan sát, hãy chạy fixture với race detector:

```
go test -race -tags raceexercise ./broken
```

Lệnh này được mong đợi là thất bại và in `WARNING: DATA RACE`. Đó là một failure có chủ ý của lab, không phải một check được phép xanh cho milestone. Race detector ghi stack trace ở các access xung đột và nơi các goroutine liên quan được tạo. Khi đọc report, hãy lần theo ba câu hỏi: biến nào được dùng chung, access nào là write, và synchronization nào đáng lẽ

phải tạo thứ tự giữa chúng?

Phần report	Nó trả lời gì	Đừng kết luận quá mức
Read by goroutine hoặc Write by goroutine	Access hiện tại nằm ở dòng nào?	Dòng đó chưa chắc là nguồn gốc policy sai.
Previous ... by goroutine	Access xung đột nào chồng lên nó?	Thứ tự in ra không phải timeline đầy đủ của chương trình.
created at	Goroutine nào mở đường cho access này?	Một report không chứng minh mọi đường chạy đều racy.

Bảng 5 — Ba phần hữu ích nhất của một race report

Detector là công cụ tìm bằng chứng khi code đã chạy, không phải chứng minh tuyệt đối rằng chương trình không có race. Một đường chưa được test chưa tạo memory access để detector quan sát. Vì thế `go test -race` đứng cạnh test tốt và workload có ý nghĩa; nó không thay thế chúng.

Sửa invariant, không chỉ làm detector im lặng

Trước khi thêm `sync.Mutex`, hãy nói invariant bằng một câu: `Successes` phải trả về số lần `RecordSuccess` đã hoàn tất, kể cả khi nhiều worker đang chạy. `Counter` vì thế mang một field `mu sync.Mutex` cạnh `successes`. `Lock` không còn là bùa chú; nó là ranh giới bảo vệ hai access vào state dùng chung:

```
func (c *Counter) RecordSuccess() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.successes++
}

func (c *Counter) Successes() int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.successes
}
```

Phiên bản này nằm trong `labs/part6-race-detector/fixed`. Test của nó chờ mọi worker bằng `sync.WaitGroup`, sau đó kiểm tra `count` là 128; `go test -race ./fixed` phải xanh. `WaitGroup` ở đây chỉ giúp test biết khi nào worker xong. Nó không tự bảo vệ `successes`; mutex mới làm việc đó. Phân biệt hai trách nhiệm này ngay từ đầu sẽ tránh rất nhiều “đã Wait rồi, sao vẫn race?” sau này.

Mutex và memory ordering sẽ được đào sâu ở phần concurrency. Ở Chương 6, bài học hẹp hơn: một test có assertion đúng vẫn có thể chưa quan sát được cách state bị chạm. Nếu requirement đưa goroutine vào code, thêm `-race` vào bằng chứng chấp nhận thay đổi là một quyết định testability, không phải một nghi thức trang trí.

Bài điều tra. Chạy fixture broken một lần với `-race`, rồi mở `fixed/counter_test.go`. Trước khi chạy phiên bản fixed, khoanh hai method cùng chạm `successes` và dự đoán vì sao cả getter cũng lock. Sau đó thử bỏ lock riêng trong `Successes`; test hiện tại vẫn có thể xanh vì nó đọc sau `wg.Wait`, nhưng public method đã mất contract an toàn cho caller đồng thời.

Đáp án — chỉ đọc sau khi đã tự làm. `RecordSuccess` ghi còn `Successes` đọc cùng field, nên cả hai cần đi qua cùng mutex nếu API hứa dùng được đồng thời. `wg.Wait` chỉ tạo điểm chờ trong test hiện tại; nó không thay thế synchronization trong API. Fixture broken được tag riêng để repository vẫn có suite mặc định xanh, còn lệnh `race` được chạy như một thí nghiệm mà failure là kết quả đúng.

Fuzz khi property đã rõ

Các subtest của config đã có tên cho những biên mà ta chủ động chọn: port 0, host rỗng, port quá lớn. Nhưng parser nhận một `string`; không ai có thể liệt kê trước mọi tổ hợp dấu ngoặc, Unicode, dấu hai chấm hay byte lạ mà input từ environment có thể mang theo. Fuzzing giúp tạo thêm input, nhưng chỉ có giá trị khi target biết phải đòi điều gì ở mọi input đó.

Property của `LoadTargets` không phải “mọi string đều được parse”. Phần lớn string phải bị từ chối. Property đúng là: nếu `LoadTargets` trả thành công, nó trả đúng một target có host không rỗng và port trong `1..65535`. Điều này ghép tốt với table test: table nói các case quan trọng nào phải bị từ chối hay được map đúng; fuzz tìm xem còn input nào làm implementation tự mâu thuẫn với invariant đó.

Đừng thay property ấy bằng “function không panic”. Không panic là một hygiene check đáng có, nhưng một parser có thể bình tĩnh trả `Target{Host: "", Port: 0}` và vẫn phá command ở chỗ xa hơn. Property mạnh hơn buộc test nhìn vào kết quả thành công. Ngược lại, đừng bắt fuzz phải đoán error message hay bắt mọi input xấu rơi đúng vào một nhánh implementation; những điều đó làm target giòn mà không tăng sự hiểu biết về contract.

Trong thực tế có nhiều họ property: kết quả luôn giữ invariant, encode rồi decode trả về dữ liệu tương đương, hay hai cách biểu diễn phải đồng thuận. Ta chỉ dùng họ đầu tiên ở đây vì parser này chưa có format output để round-trip. Chọn property theo API đang có giúp fuzzing phục vụ thiết kế, thay vì ép mọi function phải có một bài test ngẫu nhiên.

```

func FuzzLoadTargetsNeverReturnsInvalidTarget(f *testing.F) {
    for _, seed := range []string{
        "",
        "billing.internal:8443",
        "[2001:db8::10]:443",
        "payments.internal:0",
        "not a target",
    } {
        f.Add(seed)
    }

    f.Fuzz(func(t *testing.T, raw string) {
        targets, err := LoadTargets(lookup(raw))
        if err != nil {
            return
        }
        if len(targets) != 1 {
            t.Fatalf("got %d targets", len(targets))
        }
        target := targets[0]
        if target.Host == "" || target.Port < 1 || target.Port > 65535 {
            t.Fatalf("invalid target %+v", target)
        }
    })
}

```

`f.Add` không phải danh sách đầy đủ. Nó là seed corpus có chủ đích: một default, một hostname thường, một IPv6, một range error và một chuỗi không có hình dạng target. Khi chạy `go test`, seed corpus này vẫn được chạy như test bình thường. Khi chạy với `-fuzz`, toolchain mutate corpus để tìm input tạo coverage mới. Nếu tìm được failure, Go lưu input tái lập được để lần `go test` sau không làm bug biến mất theo may rủi.

Từ `labs/part6-testable-command`, chạy một lượt có giới hạn thời gian:

```

go test -run=^$ `
  -fuzz=FuzzLoadTargetsNeverReturnsInvalidTarget `
  -fuzztime=2s ./internal/config

```

-run=^\$ bỏ qua ordinary test trong lượt fuzz riêng này; chúng vẫn phải chạy trong suite bình thường. -fuzztime=2s là ngân sách cho thí nghiệm local, không phải một con số chứng nhận parser an toàn. Fuzz target xanh chỉ nói rằng các input đã chạy chưa phá property; nó không thay thế review, test case có ý nghĩa hay validation giới hạn tài nguyên.

Bài thiết kế property. Một đề xuất nghe có vẻ hợp lý là “nếu `err == nil`, `target.Name` luôn là `billing`”. Hãy quyết định có nên đưa nó vào fuzz target này không. Nếu ngày mai config hỗ trợ nhiều service, assertion đó sẽ làm test cần refactor dù parser vẫn đúng. Hãy giữ property gần với contract hiện tại nhất: shape hợp lệ của target, không phải một chi tiết của demo.

Đáp án — chỉ đọc sau khi đã tự làm. `Name == "billing"` là contract hiện tại của fixture nhỏ, nhưng không phải property parser cần giữ khi input model lớn hơn. Fuzz property nên đủ mạnh để bắt target lỗi và đủ hẹp để không đóng băng một thiết kế chưa được hứa. Khi fuzz tìm được failure thật, trước hết biến input đó thành một regression test dễ đọc; corpus là bằng chứng, test có tên là lời giải thích cho người đến sau.

Đo câu hỏi trước khi tối ưu câu trả lời

Sau khi thấy parser có nhiều validation, một phản xạ quen thuộc là đề nghị cache kết quả: “`net.SplitHostPort` có thể chậm”. Nhưng `LoadTargets` hiện chỉ chạy khi command khởi tạo. Kể cả một benchmark cho thấy override tốn allocation, điều đó chưa biến nó thành bottleneck của một CLI chạy vài probe. Benchmark tốt bắt đầu bằng câu hỏi đủ cụ thể để kết quả có thể thay đổi một quyết định.

Ở đây câu hỏi hẹp là: một lần load default, hostname override và IPv6 override có chi phí tương đối ra sao trên máy đang chạy? Test setup nằm ngoài timer; `b.N` do framework chọn để lấy mẫu đủ lâu, còn `b.ReportAllocs` yêu cầu output thêm allocation:

```

func BenchmarkLoadTargets(b *testing.B) {
    tests := []struct {
        name string
        raw  string
    }{
        {name: "default"},
        {name: "hostname override", raw: "payments.internal:9443"},
        {name: "IPv6 override", raw: "[2001:db8::10]:443"},
    }

    for _, tt := range tests {
        b.Run(tt.name, func(b *testing.B) {
            lookup := func(string) (string, bool) {
                return tt.raw, tt.raw != ""
            }
            b.ReportAllocs()
            b.ResetTimer()
            for i := 0; i < b.N; i++ {
                _, _ = LoadTargets(lookup)
            }
        })
    }
}

```

Chạy từ lab với `go test -run=^$ -bench=BenchmarkLoadTargets -benchmem ./internal/config -run=^$` tránh trộn ordinary test vào lượt đo; `-benchmem` cho `allocs/op` và `B/op`. Đừng copy số `ns/op` của máy này vào một README như một sự thật phổ quát. CPU, Go version, power mode và noise của hệ thống đều làm số đổi. Nếu một thay đổi performance thật sự sắp được nhận, hãy đo trước/sau trên cùng môi trường, lặp lại và ghi đúng workload mà quyết định đang phục vụ.

Trong case này, kết luận có thể là “không tối ưu”. Đó là một kết quả kỹ thuật hoàn toàn hợp lệ: config parse một lần chưa đáng đổi API, thêm cache hay làm code kém đọc. Benchmark đã làm việc của nó khi giúp ta từ chối một tối ưu không có pressure, chứ không chỉ khi nó dẫn tới một patch nhanh hơn.

Bài review. Chạy benchmark hai lần. Chỉ đề xuất tối ưu khi có workload như config reload trong hot loop hoặc profile thật; “số lớn hơn” chưa đủ.

Contract ở ranh giới process

Unit test của `app.Run` cố ý không biết terminal hay exit code. Điều đó giúp nó giữ boundary hẹp, nhưng cũng tạo một khoảng trống cần kiểm tra ở đúng nơi: command vẫn phải in success vào stdout, đưa config error sang stderr và trả mã exit 2. Đây là contract nhìn thấy được của process; đổi nó có thể làm script của operator hỏng dù các package phía trong vẫn xanh.

Gọi `main` trong test thường làm khoảng trống ấy khó kiểm chứng, vì `os.Exit` kết thúc process test. Thay vì fake cả process, ta rút phần presentation thành một function nhỏ nhận writer và trả exit code. `main` chỉ nối dependency thật vào function đó:

```
func main() {
    runner := func(context.Context, probe.Endpoint) error {
        return nil
    }
    os.Exit(run(
        context.Background(),
        os.Stdout,
        os.Stderr,
        os.LookupEnv,
        runner,
    ))
}
```

`run` vẫn gọi `app.Run`; nó chỉ thêm policy process rất nhỏ. Test vì thế có thể dùng `bytes.Buffer` làm stdout và stderr, rồi quan sát đúng ba thứ mà shell nhìn thấy: code, output thành công và output lỗi.

```

func TestRunWritesSuccessContract(t *testing.T) {
    var out, errOut bytes.Buffer
    code := run(
        context.Background(),
        &out,
        &errOut,
        func(string) (string, bool) { return "", false },
        func(context.Context, probe.Endpoint) error { return nil },
    )

    if code != 0 {
        t.Fatalf("code = %d", code)
    }
    if out.String() != "billing healthy=true\n" {
        t.Fatalf("stdout = %q", out.String())
    }
    if errOut.Len() != 0 {
        t.Fatalf("stderr = %q", errOut.String())
    }
}

```

Test này không assert run loop outcome thế nào hay app.Run parse config ra sao. Những contract đó đã thuộc package riêng. Nó chỉ giữ lời hứa của command: success đọc được bằng máy trên stdout và không rò error message.

Test phía failure cũng phải hẹp như vậy. Đưa vào một port không hợp lệ, để runner làm test fail nếu bị gọi, rồi assert exit 2, stdout rỗng và diagnostic ở stderr. Đừng so toàn bộ message: wording validation có thể được cải thiện, còn nơi xuất hiện và exit policy mới là contract vận hành ổn định.

Bài code review. Chuyển một `fmt.Fprintf` từ `out` sang `errOut` rồi quyết định đó là chỉnh presentation vô hại hay breaking change cho shell pipeline. Sau đó tưởng tượng một mode `--json`: contract test nên có case riêng cho mode được hứa rõ ràng này, không được làm contract text mode yếu đi đến mức nhận bất cứ output nào.

Đáp án — chỉ đọc sau khi đã tự làm. Trong shell pipeline, stdout thường mang dữ liệu command sau sẽ tiêu thụ; chuyển healthy result sang stderr có thể là breaking change. Test không cố đóng băng dấu câu mãi mãi. Nó bảo vệ boundary mà con người và automation dựa vào: success exit 0 và có result trên stdout; configuration failure exit 2, không in success data, đồng thời có diagnostic ở stderr.

Contract test này không thay thế một end-to-end test khởi động binary thật. Khi sau này command nhận flag, file config, signal hoặc network thật, một số luồng cần được kiểm chứng qua process thật. Nhưng dùng binary test để assert mọi dấu cách và mọi nhánh validation sẽ chậm, khó đọc và trùng với unit test. Ranh giới tốt là: logic có dependency thay thế được kiểm tra ở package bên trong; vài lời hứa mà process công bố được giữ ở run; chỉ một số đường đi quan trọng mới cần vượt ra integration test.

Vì vậy, việc tách run không phải là đổi thiết kế chỉ để làm test pass. Nó làm chính sách vốn đã tồn tại trong main có tên, input và output rõ. Code review có thể hỏi một câu cụ thể: “thay đổi này có làm automation nhận stdout, stderr hoặc exit code khác không?” Nếu có, tác giả phải chủ động quyết định đó là compatibility change, chứ không để nó lọt qua dưới vỏ bọc refactor nội bộ.

Đây là điểm dừng của mốc hiện tại: testability không đồng nghĩa phủ một lớp mock lên mọi package. Nó là khả năng đặt từng policy vào một boundary đủ nhỏ để ta tạo input, quan sát kết quả và biết chính xác thay đổi nào đang được bảo vệ.

Chương 7 — Dữ liệu đi vào và đi ra

Một lệnh vừa in ra một dòng `billing healthy=true` có vẻ như đã giao tiếp được với thế giới bên ngoài. Nhưng `terminal` chỉ là một `io.Writer` rất đặc biệt. Khi dữ liệu vào chuyển thành tệp cấu hình, pipe từ lệnh khác, body HTTP hay một stream nén, trực giác “đọc file rồi có dữ liệu” bắt đầu che một sự thật quan trọng: dữ liệu không nhất thiết đến cùng lúc, và tài nguyên không tự hết hạn đúng lúc ta muốn.

Mô hình tinh thần của chương này là **một stream là lời hứa về tiến độ, không phải một slice đã nằm sẵn trong bộ nhớ**. Reader hứa sẽ đưa byte theo từng lượt; người gọi phải xử lý từng lượt, biết khi nào dữ liệu kết thúc và biết ai chịu trách nhiệm đóng tài nguyên. Từ đó JSON, file và output không còn là các API rời rạc.

Một lần Read không có nghĩa là toàn bộ input

Hãy bắt đầu với một lỗi nhỏ hơn `opsprobe`. Một cấu hình JSON trông như thế này:

```
[
  {
    "name": "billing",
    "endpoint": "https://billing.internal/health"
  }
]
```

Một cách nghĩ sai nhưng rất tự nhiên là cấp buffer đủ lớn cho mẫu test, gọi `Read` một lần, rồi phân tích số byte nhận được. Nó thường xanh với `strings.NewReader` và input ngắn. Nhưng `io.Reader` không hứa lấp đầy buffer. Reader có thể là file, socket, pipe hay decoder khác; mỗi lần gọi chỉ nhận được một phần byte hiện có.

Contract tối thiểu của `Read(p)` là: nó trả `n` byte nằm trong `p[:n]`, và có thể trả một error. `n` có thể nhỏ hơn `len(p)`. Đặc biệt, một reader được phép trả byte hữu ích cùng `io.EOF` trong một lượt; caller phải tiêu thụ `p[:n]` trước khi xử lý error. `io.EOF` nghĩa là không còn byte nào cho stream đó, không

phải là “lần gọi trước không có dữ liệu”.

Kết quả	Caller phải làm gì	Diễn giải đúng
<code>n > 0, err == nil</code>	Xử lý <code>p[:n]</code> , rồi đọc tiếp	Stream đang tiến lên nhưng chưa kết thúc.
<code>n > 0, err == io.EOF</code>	Vẫn xử lý <code>p[:n]</code> , rồi kết thúc	Lượt cuối vừa có dữ liệu vừa báo hết stream.
<code>n == 0, err == io.EOF</code>	Kết thúc	Không còn byte để tiêu thụ.
<code>n == 0, error khác nil</code>	Trả hoặc phân loại lỗi	Reader không tạo được byte kế tiếp.

Bảng 6 — Cách đọc kết quả Read

Đừng tự viết vòng Read chỉ để chứng minh đã hiểu contract nếu standard library đã có primitive đúng. `io.ReadAll` là lựa chọn hợp lý khi input có giới hạn kích thước đáng tin và việc giữ toàn bộ trong memory là chấp nhận được. Khi input có thể lớn hoặc vô hạn, thiết kế phải chuyển sang xử lý dần từng phần. Khác biệt này không phải micro-optimization: nó quyết định chương trình có thể bị ép allocate bao nhiêu memory bởi input không tin cậy.

Giới hạn byte trước khi gọi parser

“Tập cấu hình chỉ nhỏ” không phải là một giới hạn. Nếu chương trình nhận một đường dẫn hoặc reader từ bên ngoài, nó cần đặt giới hạn ngay tại ranh giới mình kiểm soát. Chỉ đặt giới hạn sau `json.Decode` là quá muộn: bộ phân tích đã có quyền đọc bất cứ lượng byte nào trước khi chính sách được áp dụng.

Trước khi mở `bounded.go`, chỉ đọc hai test `TestReadBounded...` Chúng đòi `ReadBounded` nhận tối đa `max` byte; input lớn hơn đúng một byte phải trả error mà caller nhận diện được bằng `errors.Is`. Hãy tự viết phần thân theo ba bước: bọc reader để chỉ cho phép tối đa `max + 1` byte, đọc phần đã bọc, rồi so độ dài với `max`. Byte thứ `max + 1` không phải dữ liệu cần giữ; nó là bằng chứng phân biệt tệp vừa chạm ngưỡng với tệp thật sự vượt ngưỡng.

Lab đầy đủ còn từ chối `max < 0` và thêm ngữ cảnh cho error. Những chi tiết đó cần có trong code chạy thật, nhưng chúng không được che câu hỏi đang học: vì sao phải xin thêm đúng một byte.

`io.LimitReader` chỉ giới hạn số byte mà reader bọc ngoài sẽ trả; nó không tự nói JSON có hợp lệ hay không, và cũng không làm input nhỏ trở nên đáng tin. Vì thế giới hạn byte là một chính sách tài nguyên riêng; `Decode` vẫn chịu trách nhiệm cho cấu trúc JSON và số tài liệu. Hai test ngăn bảo vệ hai câu hỏi khác nhau, không gộp chúng thành một assertion mơ hồ kiểu “cấu hình lỗi”.

Đây cũng là điểm cần tránh một lời hứa sai về cancellation. `io.Reader` chỉ công bố `Read`; interface không mang `context.Context`. Một wrapper như `ReadBounded` có thể dừng sau khi nhận đủ byte, nhưng không tự làm một underlying reader đang block thức dậy. Với file local, điều đó thường không phải policy cần thêm ở đây. Với body HTTP, deadline và cancellation thuộc lifecycle request sẽ được dạy lại ở chương networking, nơi ta nhìn được owner của connection và cách transport phản ứng.

Failing test trước parser

Lab `labs/part7-stream-boundaries` không gắn ngay tệp JSON vào `opsprobe`. Project xuyên suốt hiện chưa cần tệp cấu hình; nhét nó vào lúc này chỉ khiến trực giác mới bị lẫn với chính sách endpoint cũ. Ta dùng một chương trình tái hiện tối thiểu để thấy ranh giới stream trước.

Mở `targets/targets_test.go` và chỉ đọc

`TestDecodeConsumesChunkedStream`. `chunkReader` cố tình chỉ nhả ba byte trong mỗi lần gọi. Trước khi xem `Decode`, hãy trả lời: nếu implementation giả định một lượt `Read` là đủ, test sẽ mất đoạn nào của document? Sau đó tạm đổi tên `Decode`, chạy test và tự dựng lại function theo contract mà test đòi.

Function kết thúc có thể nhỏ:

```

func Decode(r io.Reader) ([]Target, error) {
    dec := json.NewDecoder(r)
    var targets []Target
    if err := dec.Decode(&targets); err != nil {
        return nil, fmt.Errorf(
            "decode target document: %w", err,
        )
    }

    var extra any
    if err := dec.Decode(&extra); !errors.Is(err, io.EOF) {
        if err == nil {
            return nil, errors.New(
                "target config must contain one JSON value",
            )
        }
        return nil, fmt.Errorf(
            "read after target document: %w", err,
        )
    }
    return targets, nil
}

```

`json.Decoder` nhận một `io.Reader`, nên nó tự tiếp tục đọc qua nhiều chunk; caller không cần giả vờ socket hay file là slice. Test thứ hai đặt thêm một JSON value sau document đầu. Chỉ `Decode` một lần là chưa đủ contract: program sẽ im lặng bỏ qua dữ liệu còn lại. Lần `Decode` thứ hai không phải để lấy config mới, mà để chứng minh sau document hợp lệ chỉ còn whitespace và EOF.

Điều này không biến JSON decoder thành lời giải cho mọi format. CSV có record và quoting riêng; line-oriented log có boundary theo newline; body HTTP có lifecycle mạng và limit khác. Chúng sẽ được dạy ở context phù hợp. Ở đây chỉ giữ một câu hỏi trung tâm: input kết thúc ở đâu, và code nào đã xác nhận điều đó?

Đóng output cũng là một phần của result

Khi đọc file, `defer file.Close()` gần acquisition giúp không rò file descriptor. Khi ghi file, `Close` còn có thể là nơi buffer được flush và failure cuối cùng lộ ra. Nếu function ghi JSON chỉ trả error của `Encode`, caller có thể báo thành công trước khi biết output có được đóng hoàn chỉnh hay không.

Lab đầy đủ mở resource rồi đặt `defer Close` ngay sau acquisition. Để nhìn riêng policy chọn result, phần lỗi có thể viết ngắn hơn:

```
func save(w io.WriteCloser, targets []Target) error {
    writeErr := Encode(w, targets)
    closeErr := w.Close()
    if writeErr != nil {
        return writeErr
    }
    return closeErr
}
```

Lab đầy đủ vẫn đóng tài nguyên trên mọi đường return. Đoạn trên chỉ làm policy lộ ra: giữ write error trước, rồi mới trả close error. Writer giả giữ contract ấy mà không cần filesystem thật.

Tự kiểm tra trước khi tích hợp

Chạy từ `labs/part7-stream-boundaries`:

```
go test ./...
go vet ./...
```

Thử lần lượt: bỏ lượt `Decode` thứ hai; cho `chunkReader` nhả một byte; rồi cho writer giả fail ở `Write` trước `Close`. Dự đoán test nào vỡ.

Đáp án — chỉ đọc sau khi đã tự làm. Bỏ lượt decode thứ hai làm `TestDecodeRejectsAdditionalDocument` thất bại; chunk nhỏ hơn không đổi contract. Khi write và close cùng fail, phải giữ write error.

Với limit, input dài đúng max byte được trả nguyên vẹn; input dài `max + 1` trả error giữ identity `ErrDocumentTooLarge`. Đó là lý do lab không dùng một `if len(data) == max` mở hồ: bằng chứng cần phân biệt đúng ngưỡng với vượt ngưỡng.

Chương này không thêm file config vào `opsprobe`: chưa có yêu cầu vận hành nào bắt nó phải có. Đó là giữ teaching vehicle phục vụ bài học. Khi một yêu cầu thật cần `import/export target`, `stream boundary` và `close policy` ở lab này sẽ là nền để tích hợp có chủ đích.

Trước khi chạm một API I/O mới, hãy tự hỏi ba câu. Byte đến theo từng lượt nào, đâu là dấu kết thúc được chấp nhận, và ai quan sát failure của resource sau cùng? Ba câu này không thay thế tài liệu của format hay protocol cụ thể, nhưng chúng ngăn một sai lầm rất phổ biến: coi input/output là vài dòng plumbing nằm ngoài contract của chương trình.

Phần tiếp theo sẽ đặt một áp lực khác lên chương trình: nhiều goroutine cùng sống trong một process. Trước khi chọn channel hay mutex, ta cần nhìn một race không phải như một câu thần chú về thread safety, mà như hai access không có thứ tự an toàn.

Chương 8 — Một race bắt đầu từ đâu

Một chương trình kiểm tra nhiều endpoint thường có một người thu thập kết quả và một người trình bày. Bản chạy thử có thể luôn in đúng trên laptop, nên ta rất dễ gọi nó là “chạy song song được”. Nhưng kết quả đúng trong một lượt không tạo ra một lời hứa đúng cho lượt kế tiếp.

Mô hình tinh thần của chương này là **race không phải là hai goroutine cùng tồn tại; race là các lần chạm vào cùng dữ liệu mà không có thứ tự an toàn giữa chúng**. Trước khi chọn channel, mutex hay atomic, ta phải lần được những lần truy cập đó và hỏi: thứ tự nào thực sự được thiết lập?

Một lá cờ không tự công bố kết quả

Hãy tưởng tượng collector chuẩn bị một báo cáo, rồi đặt cờ ready để người in biết đã xong:

```
type summary struct {
    completed int
    last      string
}

var latest summary
var ready bool

func collect() {
    latest = summary{completed: 1, last: "billing"}
    ready = true
}

func show() summary {
    for !ready {
    }
    return latest
}
```

Nguồn trông có một thứ tự hiển nhiên: gán latest, rồi mới gán ready. Nhưng đó chỉ là thứ tự **bên trong** collect. show là một luồng thực thi khác. Nó vừa đọc ready khi collect có thể ghi ready, vừa đọc latest khi

`collect` có thể ghi `latest`. Không có thao tác đồng bộ nào nổi hai bên.

Vì vậy ở đây không chỉ có một vấn đề “đôi khi in cũ”. Có hai cặp truy cập tranh chấp: đọc/ghi `ready` và đọc/ghi `latest`. Việc vòng lặp có tình cờ kết thúc ở một máy không biến các cặp ấy thành có thứ tự. Đặc biệt, nhìn thấy `ready == true` không là bằng chứng ngôn ngữ rằng người đọc cũng nhìn thấy bản `latest` đã chuẩn bị xong.

Bốn tầng không được trộn lẫn

Tầng	Câu hỏi nó trả lời	Không được suy ra thêm
Nghĩa của ngôn ngữ	Hai lần truy cập thường, cùng vị trí, có ít nhất một lần ghi và không có quan hệ happens-before là race.	Bộ chạy của máy này sẽ xen kẽ ở đúng chỗ nào.
Hành vi được tài liệu cam kết	Lệnh go khởi động goroutine mới; nhưng goroutine kết thúc không tự tạo quan hệ đồng bộ cho phần còn lại của chương trình.	Một cờ bool tự công bố mọi lần ghi xảy ra trước nó.
Chi tiết hiện thực	Bộ lập lịch, số P và thời điểm ngắt để chuyển việc quyết định một lần chạy thực tế xen kẽ ra sao.	Đó là hợp đồng ổn định của Go hay của code.
Hành vi đo được	<code>go test -race</code> báo những lần truy cập mà lượt chạy đã thực sự đi qua.	Không có báo cáo chứng minh mọi đường chạy đều không có race.

Bảng 7 — Bốn loại bằng chứng khi điều tra race

Hai tầng đầu là lời hứa để thiết kế. Tầng thứ ba chỉ hữu ích khi giải thích một vết chạy hoặc một vấn đề hiệu năng; dùng dùng nó để biện minh rằng race “khó xảy ra”. Tầng cuối là bằng chứng thực nghiệm mạnh cho đường đã chạy, nhưng nó không thay thế lập luận về thứ tự. Một test xanh không cho phép ta đổi “chưa quan sát thấy” thành “được bảo đảm”.

Vẽ chứng minh thay vì đoán lịch chạy

Chương 6 đã dùng race detector để tìm một lần truy cập thực sự xung đột. Ở đây bước kế tiếp không phải chạy lại cùng báo cáo, mà là viết được lời giải thích cho phiên bản đúng trước khi chạy. Ta sẽ dùng ba mũi tên:

- **sequenced-before**: thứ tự của code trong một goroutine. `append` xong rồi mới `Done` là một mũi tên loại này.
- **synchronized-before**: mũi tên mà ngôn ngữ hoặc API công bố. Ví dụ `Mutex.Unlock` và một `Mutex.Lock` về sau, hoặc `Done` và `Wait` mà nó mở khóa.
- **happens-before**: đường đi tạo bởi hai loại mũi tên trên. Nếu lần ghi đi đến lần đọc qua đường này, người đọc có một lý do được cam kết để thấy giá trị đó.

Đừng biến ba tên thành câu thần chú. Với mỗi kết quả cần đọc, hãy vẽ đường cụ thể. Trong ví dụ `ready`, không có cạnh nào từ `latest = ...` sang việc `show` đọc `latest`; vì vậy lời giải không thể chỉ là thêm một vòng lặp chờ.

Một khóa bảo vệ một bất biến, không bảo vệ một dòng

Một `Mutex` hợp với dữ liệu thật sự được sở hữu chung, khi nhiều goroutine cần lần lượt sửa hoặc chụp lại cùng một trạng thái. Bất biến của lab mới là: **sau khi người điều phối đã chờ tất cả worker hoàn tất, snapshot chứa đúng một report của mỗi worker**. `Mutex` bảo vệ `slice reports` trong lúc thêm và sao chép; `WaitGroup` cho người điều phối biết khi nào có thể đánh giá bất biến.

Trước khi mở `fixed/registry.go`, chỉ đọc `exercise/registry_test.go`. Test đang đòi một `Registry`, một `Add(Report)` và một `Snapshot() []Report`. Hãy tự quyết định field nào cần có, method nào dùng pointer receiver, và vùng lock cần bao trùm đến đâu. Lệnh khởi đầu cố ý đỏ vì phần hiện thực chưa tồn tại:

```
go test -tags exercise ./exercise
```

Đừng đáp bằng cách trả slice nội bộ. Bài cũ về aliasing đã cho ta lý do: người gọi giữ slice trả về có thể thay phần tử của nó. Snapshot ở đây hứa một ảnh chụp của container, nên lab dùng copy để người gọi không ghi đè phần tử bên trong registry. Report chỉ có string value để câu hỏi của bài vẫn là thứ tự; quyền sở hữu sâu hơn của object lồng nhau sẽ quay lại khi nó có nhu cầu thật.

Một phần hiện thực được chấp nhận không cần biết bộ lập lịch sẽ chạy worker nào trước. Nó chỉ cần khiến mỗi Add lock trước khi append, unlock sau đó, và Snapshot lock trong lúc tạo bản sao. Tài liệu sync cam kết Unlock xảy ra trước một Lock về sau trên cùng mutex. Đây là lời hứa API, không phải suy luận từ số core hay một lần test xanh.

WaitGroup tạo mốc hoàn tất, không thay Mutex

Test đặt wg.Add(workers) trước khi tạo goroutine. Mỗi worker gọi registry.Add(report) rồi mới wg.Done() qua defer. Khi wg.Wait() trả về, mỗi Done đã mở khóa nó; do đó mọi Add trước Done đã nằm trước thời điểm assertion gọi Snapshot.

Đường chứng minh cho một worker có thể đọc thành lời:

```
Add append reports
  → Unlock registry.mu
  → Done
  → Wait returns
  → Snapshot Lock
  → copy reports
```

Mũi tên đầu, thứ hai và mũi tên sau Wait là thứ tự trong code; Done → Wait returns là điều sync.WaitGroup công bố. Mutex giữ các access vào slice không chồng lên nhau. WaitGroup không thay thế lock: nếu các worker cùng append vào một slice mà không khóa, chúng vẫn race dù main có chờ đến khi tất cả xong mới đếm kết quả.

Sau khi tự làm, chạy bản tham chiếu độc lập:

```
go test -race ./fixed
```

Kết quả xanh ở đây là hai loại bằng chứng cùng lúc: test kiểm tra bất biến cụ thể, còn race detector không thấy lần truy cập racy trên đường chạy của test. Nó vẫn không phải lời khẳng định rằng mọi API tương lai thêm vào Registry đều an toàn. Bất cứ method mới nào đọc hoặc ghi reports đều phải quay lại bất biến và phạm vi lock.

Điều go cho phép, điều nó không hứa

Lệnh `go f()` thiết lập một quan hệ từ nơi khởi động đến lúc `f` bắt đầu. Điều đó đủ để goroutine mới thấy dữ liệu đã được chuẩn bị trước lệnh `go`; nó không cho chiều ngược lại. Việc `f` return hay goroutine kết thúc không tự báo cho goroutine khác rằng công việc đã xong. Đó là lý do `WaitGroup` trong `lab` là một phần của chứng minh, không phải nghi thức để test bớt flake.

Không dùng `time.Sleep` để thay mũi tên còn thiếu. `Sleep` chỉ đoán rằng worker có đủ thời gian trên một lần chạy; nó không công bố thứ tự nào và làm test vừa chậm vừa không chắc. Cũng chưa dùng `channel` hoặc `atomic` để vá ví dụ `ready`: chúng có nghĩa riêng và sẽ được dạy đầy đủ ở Chương 9, nơi câu hỏi chuyển sang truyền công việc, quyền sở hữu và áp suất của dòng dữ liệu.

Chương này kết thúc khi anh nhìn hai goroutine không còn hỏi “có chạy cùng lúc không?”, mà hỏi “lần truy cập nào cùng dữ liệu, bất biến nào cần giữ, và đường happens-before nào làm điều đó đúng?”. Câu hỏi ấy đi cùng anh qua `channel`, `HTTP handler`, `cache` và `code runtime`; cơ chế có thể đổi, nhưng trách nhiệm chứng minh thứ tự không đổi.

Ghi chú kiểm chứng

1. Go Team. The Go Memory Model, phiên bản 6 June 2022. go.dev/ref/mem
2. Go Team. Data Race Detector. go.dev/doc/articles/race_detector
3. Go Team. Package sync. pkg.go.dev/sync